



I Executive Summary

[Summarise the whole report: state all the major points of your project, and outline the report's scope, purpose and conclusions.]



I Acknowledgements

[Optional. If used, it must come after the Executive Summary and before the Table of Contents.]



I Table of Contents

I 1 Introduction

[Briefly outline the company and/or department you worked for. Summarise the work you have done and clearly state the objectives of your project. You can also state the importance of your project and its relevance.]

I 2 Background

This section first describes the anatomy of the pulmonary airways and what current imaging can resolve of it (Section “??”), then the ways a 3D airway tree can be represented (Section “??”), and finally reviews related work (Section “??”).

II 2.1 Anatomy of the pulmonary airways

The pulmonary airways form a branching tree of tubes that carries air from the trachea to the alveoli, where gas exchange takes place (Figure ??). This subsection summarizes how the tree is organized, how its branches are named, which part of it current imaging can resolve, and how the part that cannot be imaged is estimated.

Structure and function.

Starting from the trachea, each airway divides into daughter branches of smaller diameter and length. The branching level is counted in *generations*: in the classical symmetric model of ?, the trachea is generation 0 and every airway splits into two, so that generation z contains 2^z airways and the tree ends after 23 generations (Table ??). Generations 0 to 16 form the *conducting zone*: the trachea, bronchi and bronchioles down to the terminal bronchioles. They take no part in gas exchange; they conduct, warm, humidify and filter the inspired air, and their volume forms the anatomical dead space. Generations 17 to 23 form the *respiratory zone*, whose respiratory bronchioles, alveolar ducts and alveolar sacs carry the alveoli in which oxygen and carbon dioxide diffuse between air and blood (?). Because the number of airways doubles at each generation while their diameter shrinks by only about a fifth, the total cross-section grows quickly towards the periphery and the air slows down, so that transport in the deepest generations is dominated by diffusion. The bronchi are supported by cartilage, whereas the bronchioles are not; the non-cartilaginous airways with an internal diameter below about 2 mm, found from roughly the eighth generation onwards, are called the *small airways* (?). Real airway trees are not symmetric: the two daughters of a bifurcation usually differ in diameter, length and angle, and the number of generations between the trachea and a terminal bronchiole varies from one path to another (??).

Anatomical nomenclature.

Below the generation count, airways are named after the lung region they supply. At the carina the trachea divides into the right and left main bronchi. The right lung has three lobes (upper, middle and lower) and the left lung two (upper, including the lingula, and lower), each supplied by a lobar bronchus. The lobar bronchi divide into *segmental bronchi*, each ventilating one bronchopulmonary segment. The right lung has ten segmental bronchi, RB1 to RB10; in the left lung, the apical and posterior segments and the anterior and medial basal segments usually share a bronchus (LB1+2 and LB7+8), which

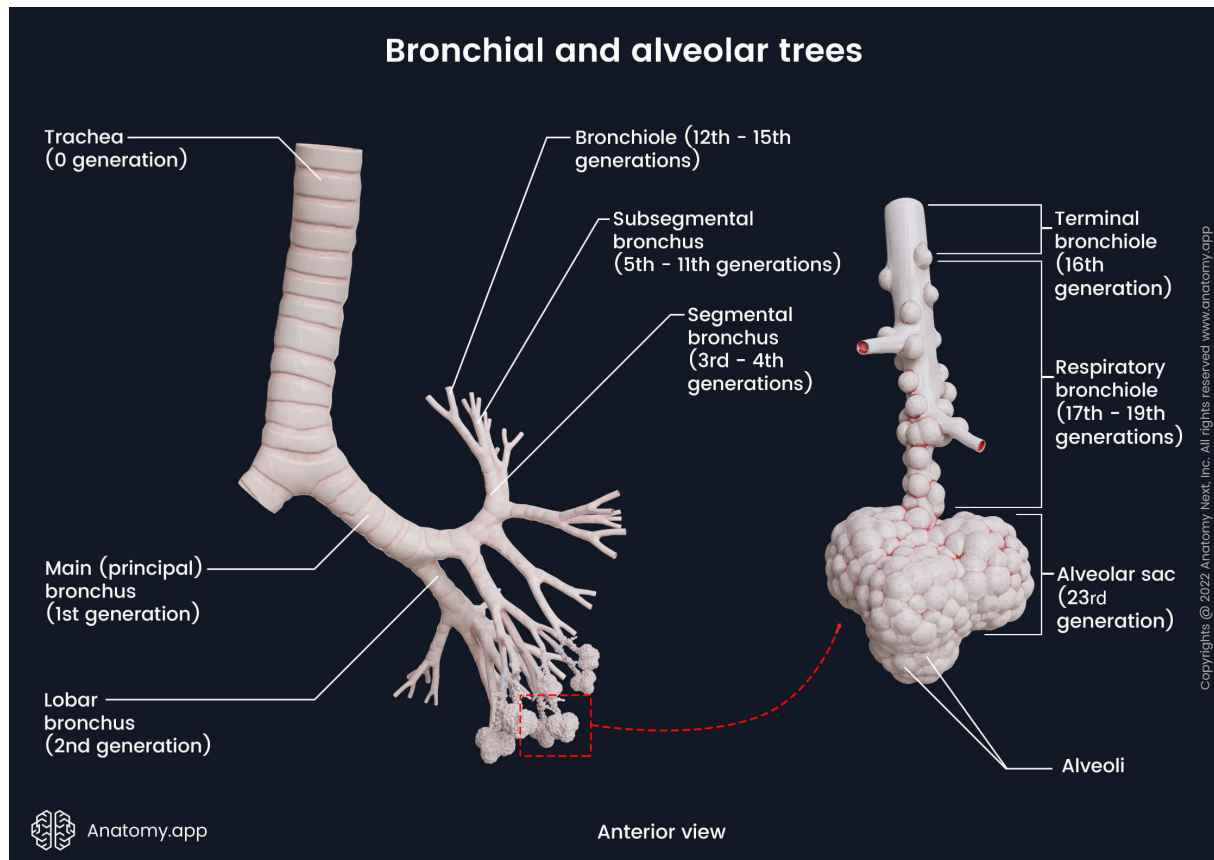


Figure 1: Bronchial and alveolar trees (anterior view). The conducting airways run from the trachea (generation 0) through the main, lobar, segmental and subsegmental bronchi and the bronchioles to the terminal bronchioles (generation 16); the enlarged detail shows the respiratory zone, with respiratory bronchioles (generations 17–19) and an alveolar sac (generation 23) covered with alveoli. The generation ranges differ slightly between sources (compare Table ??). Image © 2022 Anatomy Next, Inc., reproduced from Anatomy.app (?).

Table 1: Airway generations in the symmetric model of ?. Diameters are approximate model values for an adult lung; real trees are asymmetric, so the correspondence between generation and airway type is only indicative. The last column indicates which airways clinical CT can resolve (?).

Zone	Airway	Generation z	Diameter (mm)	Clinical CT
Conducting	Trachea	0	18	Yes
	Main bronchi	1	12	Yes
	Lobar bronchi	2	8	Yes
	Segmental bronchi	3	6	Yes
	Subsegmental bronchi	4	4.5	Yes
	Small bronchi	5–11	3.5–1.1	Partly
	Bronchioles	12–15	1.0–0.7	No
	Terminal bronchioles	16	0.6	No
Respiratory	Respiratory bronchioles	17–19	0.5	No
	Alveolar ducts	20–22	0.4	No
	Alveolar sacs	23	0.4	No

gives eight segmental bronchi (Table ??). This nomenclature goes back to systematic studies of the bronchopulmonary segments (?), which also documented how frequent anatomical variations are, for example trifurcations of the right upper lobe bronchus in almost half of the subjects (?) and of the left upper division and lower lobar bronchi in about a fifth (?). The $10 + 8 = 18$ segmental bronchi, together with the trachea and main bronchi, are the classes predicted by the labelling model used in this work (Section “??”) (?).

Table 2: Lobar and segmental bronchi of the human airway tree.

Lung	Lobe	Segmental bronchi
Right	Upper	RB1 apical, RB2 posterior, RB3 anterior
	Middle	RB4 lateral, RB5 medial
	Lower	RB6 superior, RB7 medial basal, RB8 anterior basal, RB9 lateral basal, RB10 posterior basal
Left	Upper	LB1+2 apicoposterior, LB3 anterior, LB4 superior lingular, LB5 inferior lingular
	Lower	LB6 superior, LB7+8 anteromedial basal, LB9 lateral basal, LB10 posterior basal

What current imaging can resolve.

Clinical CT has a spatial resolution of about 0.6 to 1 mm, which allows direct assessment of airways down to a diameter of about 2 to 2.5 mm; normal bronchioles are not visible (?). The datasets used in this work fall in this range, with in-plane resolutions of 0.5 to 0.92 mm and slice thicknesses of 0.45 to 1 mm for ATM’22 (Section “??”). In Weibel’s model, the airway diameter falls below 2 mm around the eighth generation (Table ??); in real, asymmetric trees this happens earlier along many paths, and in patients with COPD most sixth-generation airways of the apical and posterior basal segments are already narrower than 2 mm (?). A lumen that spans only one or two voxels is also blurred by the partial volume effect, so segmentations lose branches and suffer breakages towards the periphery (?). Ultra-high-resolution CT, with 0.25 mm slices and 1024×1024 matrices, can measure airways of 1 to 2 mm in vivo (??), and smaller airways have so far been studied mainly with histology and micro-CT on excised lungs (?). MRI, with hyperpolarized gases or ultrashort echo times, avoids ionizing radiation but has a lower resolution and reaches at most the segmental and subsegmental bronchi (??). In practice, an airway tree segmented from CT therefore contains the central airways and a limited number of further generations, only a small fraction of the tens of thousands of terminal bronchioles of the real tree.

Estimating the complete tree.

Because the peripheral airways cannot be imaged in vivo, the complete tree is estimated with models. Anatomically based models grow the tree inside the lung volume with fractal-like or volume-filling rules that reproduce morphometric data (??) (Figure ??). Patient-specific models combine both: the lobes and the central airways are segmented from CT, and the tree beyond CT resolution is generated inside each lobe, guided by the resolved airways and the lobar boundaries (???) (Figure ??). AVATREE follows the same idea and can extend a CT-derived tree to any number of generations (?) (Figure ??). The comparison between the central airways and the estimated tree in these figures shows how small the CT-visible part of the airway tree is. All the airway trees used in this work, and therefore all the representations and models of the following sections, are limited to this CT-resolved part.

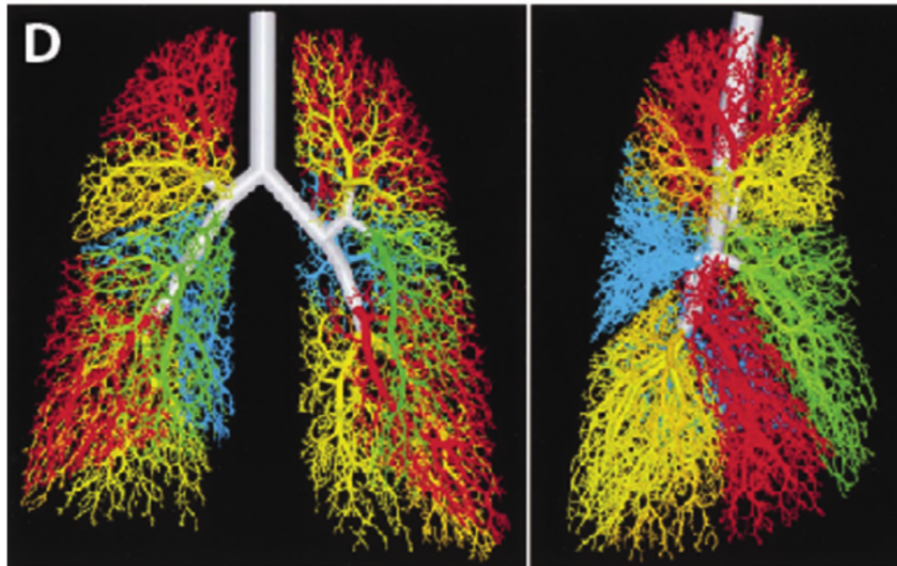


Figure 2: Three-dimensional model of the human airway tree generated with fractal-like rules, in front and side views. The model contains more than 50,000 branches and matches morphometric data from human lungs. Reproduced from ?, who adapted it from earlier work.

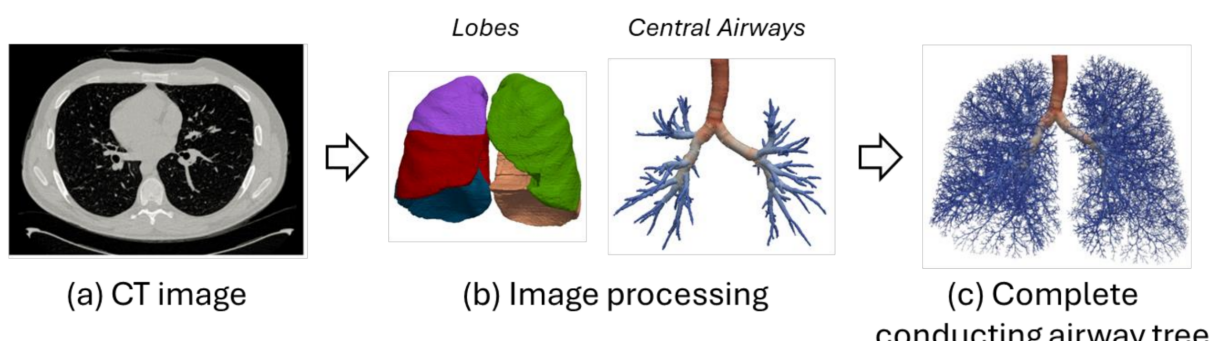


Figure 3: Patient-based model of the complete conducting airway tree: (a) CT image; (b) lobes and central airways segmented from the CT image; (c) airway tree beyond CT resolution generated with a lobe-filling algorithm, colour-coded by airway radius. Reproduced from ?, who adapted it from earlier work.

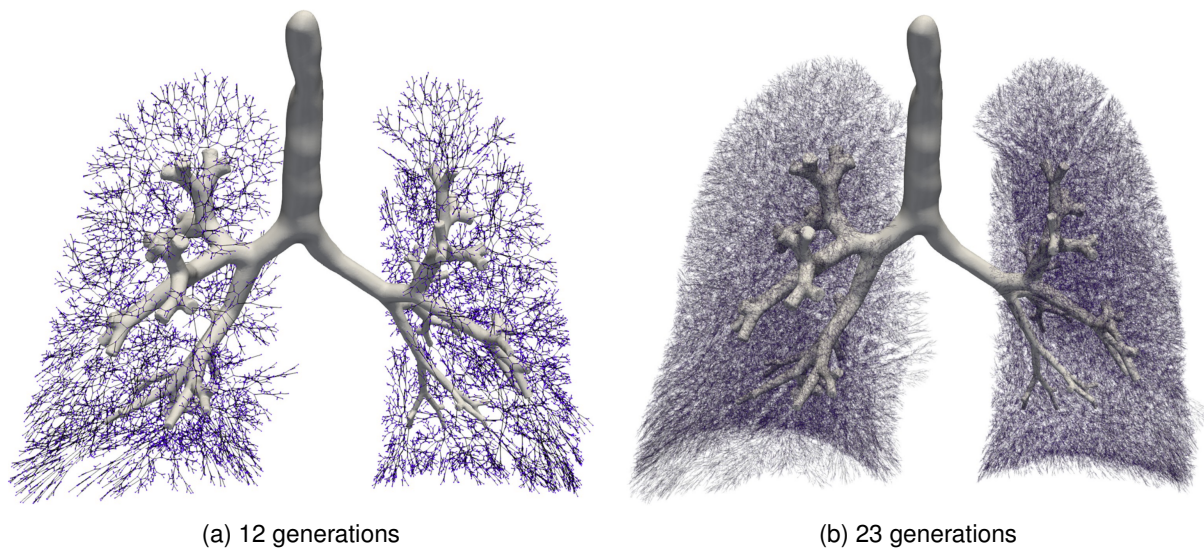


Figure 4: Bronchial tree estimated from a CT-derived airway tree up to (a) 12 and (b) 23 generations. The surface is reconstructed for the first 7 generations; deeper generations are shown as centerlines. Reproduced from ?.

2.2 Data representation

A 3D pulmonary airway tree can be stored in several ways. The choice matters as it fixes what a model sees (a volume, a surface, sample points or a skeleton), how much memory it needs, and whether the branching structure is explicit or has to be inferred. Figure ?? shows the same tree in six common representations, and Table ?? summarizes their main properties.

Voxel grid.

The airway is a binary occupancy mask on a regular 3D grid, usually the grid of the CT scan itself. This is the direct output of manual annotation and of segmentation networks, and the format in which ATM'22 and AIIIB23 are distributed (Section “??”). It fits 3D convolutional networks naturally (??), but memory grows with the cube of the resolution while almost all voxels are empty, and the shape is limited by the voxel spacing: thin distal branches cover only a few voxels and tend to look jagged or broken (?). Sparse variants store only the occupied voxels, for example with the VDB data structure (?) used by XCube (?), or with sparse convolutions (?). In this work, voxel grids are used for registration and by the dense and sparse autoencoders.

Surface mesh.

A surface mesh describes the boundary of the airway lumen with vertices and triangles. It is usually extracted from the voxel mask with Marching Cubes (?), or reconstructed from points, e.g. by Poisson surface reconstruction in AVATREE (?). Meshes are compact, give sub-voxel smooth surfaces and are the standard input for flow simulation (??), but they carry no explicit branch structure and can contain non-manifold elements, which is why their number is used as a quality metric (?). In this work, meshes are extracted as the first step of the implicit model's data preparation.

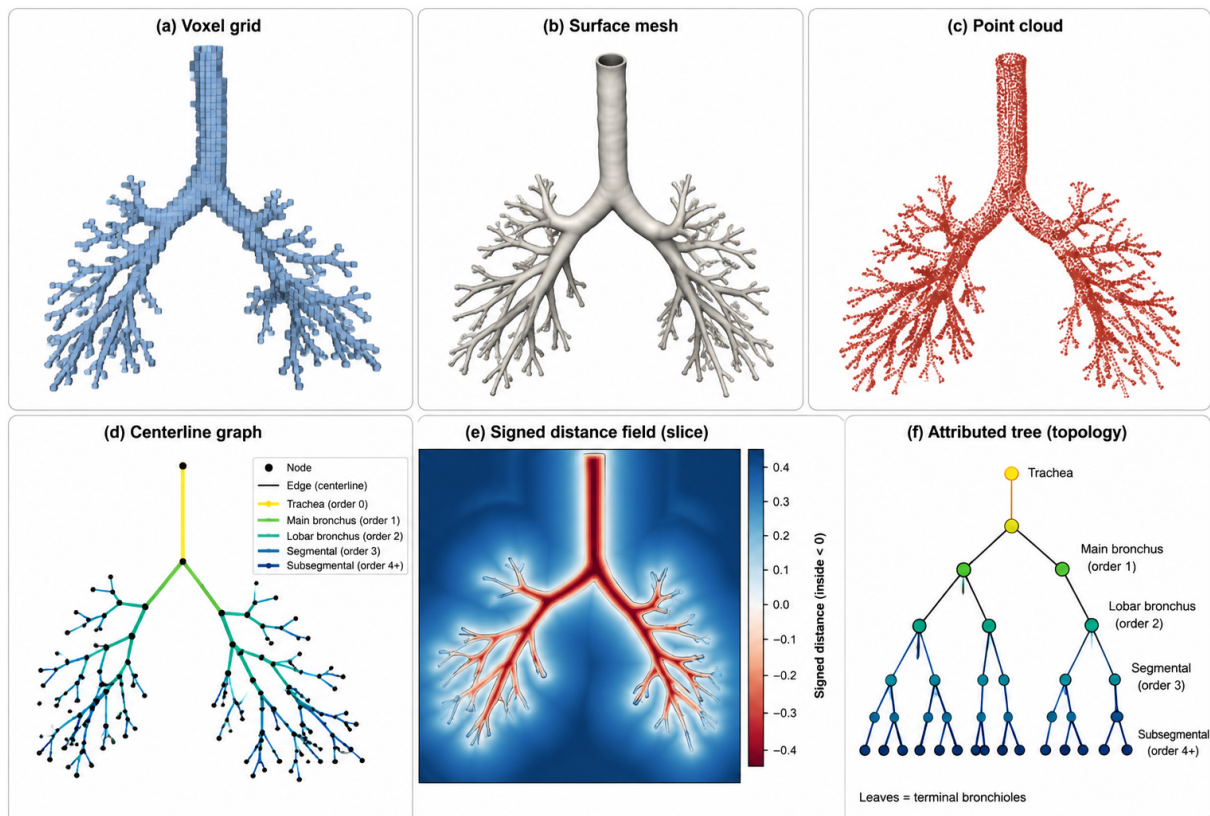


Figure 5: The same synthetic airway tree (trachea and three generations) in six representations: (a) binary voxel grid, (b) triangulated surface mesh, (c) point cloud sampled on the surface, (d) centerline graph, with edge colour and width following the branch radius, (e) signed distance field on a slice through the first bifurcation, with the airway surface as the zero level set (black line), and (f) the abstract tree topology, whose edges carry attributes such as radius r and length ℓ .

Point cloud.

A point cloud is an unordered set of 3D points sampled on the surface of the airway tree. It is sparse and independent of any grid, and it can be processed by point networks such as PointNet++ (?) and Point Transformer (?). It has no connectivity, however, so thin branches may receive too few points to be recognized.

Centerline graph.

Thinning the binary mask gives a one-voxel-wide skeleton (??), which can be converted into a graph whose nodes are bifurcations and end points and whose edges are the branches between them (?). Each edge can carry attributes such as radius, length and direction. The representation is very compact and makes the branching topology explicit, which is why it underlies most work on branch labeling (???), morphometry (??) and graph generation (?). Its weaknesses are its sensitivity to skeletonization artefacts, such as spurious short branches, and the loss of the lumen shape beyond the stored attributes. In this work, the centerline graph is an input of IPGN - the model that used to generate the labels - and the basis of the morphological descriptors.

Implicit function.

The surface can also be defined as the zero level set of a continuous function of space, typically a signed distance field (SDF) that is negative inside the airway and positive outside. Neural implicit representations learn this function with a small network conditioned on a latent code (??), or store it in feature planes that can be queried anywhere (?). The result is continuous, resolution-free and smooth, but a mesh has to be extracted to visualize it, and the topology is not explicit; DGCI addresses this by learning correspondences through a space shared by all trees (?). In this work, the implicit representation is the one that produced the best reconstructions.

Attributed tree and parametric models.

Finally, the airway can be reduced to a combinatorial tree whose edges carry geometric attributes, such as radius, length or landmark points along the branch. Idealized airway models describe the whole tree this way (??), tree-shape spaces use it to compare and average trees (??), and generative models encode it recursively (??) or as a sequence of nodes (?). Branch geometry can in turn be described parametrically, e.g. with NURBS (?) or B-spline cross-sections (?). Such representations are compact and interpretable, but they depend on a reliable extraction of the tree, and trifurcations or loops break the assumption of a binary tree (?). The morphological descriptors used in this work operate on such a tree, whose nodes carry an anatomical label, a generation number and a length.

Table 3: Main properties of the airway tree representations, and where each one is used in this work.

Representation	Geometry	Branching explicit	Used in this work
Voxel grid	Discrete volume	No	Registration, VAE, autoencoders
Sparse voxels	Occupied voxels	No	Sparse encoder
Surface mesh	Continuous surface	No	Implicit model (data preparation)
Point cloud	Sampled points	No	Labelling (IPGN), Implicit model
Centerline graph	1D skeleton	Yes	Labelling (IPGN), morphological metrics
Implicit (SDF)	Continuous field	No	Implicit encoding (DGCI)
Attributed tree	Abstract topology	Yes	Morphological metrics

II 2.3 Related work

III 2.3.1 Geometric models of the airway tree

Morphometry and idealized models.

Early airway models are based on idealized morphometry rather than on individual anatomy. Casts of human lungs were used to study branching patterns and the relation between airway lengths and diameters (?), and ? and ? proposed asymmetric models of the human bronchial tree. ? related airway diameter to branching angles, and ? examined deterministic bifurcating distributive systems with a Monte Carlo method. ? proposed a 3D model of the human airway tree in which the branching pattern is driven by a diameter–flow relationship within an organ boundary, relating branching angle to flow rate and airway length to diameter. ? gave a detailed mathematical description of airway bifurcation geometry and used it to generate surface models of idealized bifurcations, and ? presented an anatomical and functional model of the tracheobronchial tree. ? reviewed computational models of airway and pulmonary vascular structure and function towards a “Lung Physiome”.

Algorithmic generation of airway trees.

Building on the Monte Carlo approach of ?, ? introduced a volume-filling algorithm that grows a bifurcating tree inside each lung lobe. ? generated the human bronchial tree with a Lindenmayer system, and ? extended it to a stochastic parametric L-system with parallel growth of bronchopulmonary segments. ? proposed a parametric 3D model of human airways for particle drug delivery and deposition.

Subject-specific models from CT.

Subject-specific airway trees can be extracted from CT images. ? developed the skeletonization used to obtain a 1D tree from a segmented airway, and ? presented automated segmentation and morphometric analysis of the airway tree from multidetector CT. ? combined CT-based geometry analysis with finite element models of the human and ovine bronchial tree, using their volume-filling algorithm to extend the CT-resolved tree to the terminal bronchioles, guided by the resolved airways and the lobar boundaries. ? and ? combined CT-based central airways with such artificial airways to build subject-specific models from the glottis to the terminal bronchioles for CFD, but their pipeline required manual intervention to construct trifurcations; ? later reviewed multiscale image-based modeling of gas flow and particle transport in the lungs. ? reported HRCT-based airway morphometry in healthy subjects and asthmatic patients, and ? presented an algorithm for an individualized 3D deterministic model of the human

bronchial tree from HRCT images, which they evaluated statistically. ? developed a pipeline for patient-based complete conducting airway models of healthy and asthmatic subjects, covering CT acquisition, lung, lobar and airway segmentation, centerline extraction, and algorithmic generation of the distal airways beyond the CT-resolved tree.

Trifurcations are a frequent source of difficulty for these models, although they are common in real airway trees. ? studied bronchopulmonary segments in 50 subjects and reported a trifurcation at the right upper lobe bronchus in 46% of them, and ? examined the left-lung bronchial anatomy of 216 subjects with multi-detector CT and found trifurcations at the left upper division bronchus and the left lower lobar bronchus in 23% and 18% of subjects, respectively.

? proposed a centerline-based method that automatically constructs subject-specific three-dimensional models of the human airways from a CT-segmented airway skeleton and surface. Their approach systematically identifies and classifies trifurcations, distinguishing true trifurcations from double bifurcations and fork-type from tripod-type configurations, and uses dedicated geometric templates to reconstruct them. The method produces models of increasing realism (straight-centerline, curved-centerline, and fitted-surface), can be extended beyond CT resolution to the terminal bronchioles, and partitions the geometry branch by branch, which facilitates automatic mesh generation for computational fluid dynamics (CFD). Applied to 16 healthy and 16 severe asthmatic subjects, the fitted-surface models achieved an average surface error of about 11% of the image voxel size, and an aerosol deposition simulation showed branch-wise deposition efficiencies within 5% of those obtained with a CT-based model.

? presented AVATREE for generating anatomically valid patient-specific airway tree models from CT scans. Their pipeline first segments the lungs and the visible central airways and extracts the airway centerline as a graph. It then extends the tree beyond the generations visible in the image using a space-filling algorithm which splits the lung volume with PCA based planes. From the extended centerline they reconstruct a 3D airway surface suitable for CFD simulations using Poisson surface reconstruction. The framework also produces spatial probability maps of airway generations overlaid on the CT images. It can simulate broncho-constriction as seen in asthma and COPD, through Laplacian mesh contraction. The authors validated the generated trees against published morphometric data comparing Strahler and Horsfield orders, branch lengths, diameters and branching angles.

Meshing and hybrid models.

? proposed an automatic method to build and mesh tubular geometries, including trifurcations, for cardiovascular and lung applications. However, it takes a surface as input and is not centerline based, so the mesh patches are not linked to branches with anatomical data. ? developed an automatic method to generate CFD compliant triangulated meshes from segmented airways, ? introduced Snakes Isogeometric Analysis based on NURBS for healthy and COPD airway trees and ? smoothed airway surface meshes with graph convolutional neural networks. Following the same idea as ?, ? and ? built hybrid models that couple image derived upper airways with idealized lower airways for CFD.

? presented a comprehensive review of geometrical models of the human intrathoracic airways tracing their evolution from the early idealized mathematical models of the 1960s such as the symmetric Weibel model and the asymmetric Horsfield model, to modern image-based, patient-specific reconstructions derived from CT and to a lesser extent. The authors surveyed airway segmentation strategies ranging from traditional region-growing and knowledge-based methods to machine learning and deep learning approaches such as 3D U-Net architectures and outlined the pipeline required to obtain CFD compliant meshes including centerline extraction, surface smoothing and mesh generation. They also discussed hybrid models that combine CT-resolved central airways with algorithmically generated distal trees and

reviewed applications in inter-subject variability studies, diagnosis, surgical planning, drug delivery and environmental exposure. Finally, they identified key open challenges including missing small airways in segmentation, limited validation on pathological scans and the need for robust, automated mesh generation suitable for clinical use.

2.3.2 Representation learning and generative models of vascular trees

Data-driven approaches have recently emerged as an alternative to rule-based (fractal and space-filling) methods for synthesizing 3D vascular geometry. ? proposed VesselVAE, a recursive variational neural network (RvNN) that encodes vessels as binary trees where each node stores a centerline point and its radius. The encoder recursively aggregates the tree into a low-dimensional latent space while the decoder reconstructs node attributes and predicts through a node classifier, whether each node branches. New vessels are obtained by sampling the latent space and converting the generated centerlines into surface meshes. Trained on the Intra dataset the method produced samples whose radius, length and tortuosity distributions closely match real data. However, it assumes circular cross-sections and has difficulty scaling to deep trees. To overcome these limitations, the same group introduced VesselGPT (?), which treats vascular trees as sequences of nodes obtained through a preorder traversal. A VQ-VAE first learns a discrete vocabulary of node embeddings and a GPT-2 model is then trained to generate vessels autoregressively over these tokens. Instead of a single radius each cross-section is represented with a B-spline which preserves non-circular shapes such as aneurysms. The final mesh is extracted from a signed distance field using marching cubes. Evaluated on the Aneurisk dataset, VesselGPT generates deeper and more detailed vascular trees than previous recursive and diffusion-based approaches. Graph-based generation has also been explored: ?? proposed two-stage discrete diffusion models that generate simplified metric graphs of vessels and airways.

Learning compact representations of vascular structures has recently gained attention with approaches differing mainly in how the vessel geometry is represented. ? proposed VeTTA, a two-stage Transformer-based autoencoder that encodes vessel trees into a single vector representation. In the first stage a Vessel Autoencoder embeds individual vessel segments described by centerline points and radii lifted to sinusoidal Fourier features into compact latent codes. In the second stage a Vessel Tree Autoencoder encodes the full set of tree edges into a global latent vector which is decoded recursively with a slot-based Transformer decoder that guarantees a topologically valid tree. Evaluated on a synthetic 2D tree dataset and a 3D coronary artery dataset, VeTTA outperformed 3D convolutional autoencoders in reconstruction accuracy required less GPU memory and produced smooth, topologically consistent latent interpolations. However, it is restricted to tree-structured data with at most two children per node.

In contrast, ? proposed VAEsselSparse a variational autoencoder that operates directly on high-resolution organ-level binary vessel segmentation maps without assuming a tree structure. By combining sparse 3D convolutions with sparse windowed self-attention their model exploits the inherent sparsity of vascular masks and achieves an $8 \times 8 \times 8$ spatial compression while preserving vessel connectivity. VAEsselSparse outperformed a dense VAE and MedVAE on topology-aware metrics such as cDice and Betti number errors. The authors further showed that the learned latent space supports downstream tasks including aneurysm/stenosis classification, circle of Willis subtype classification and the generation of realistic vessel structures with a flow-matching model trained in the latent space.

2.3.3 Branch matching and anatomical labeling of airway trees

Registration, branch matching and branch labeling of airway trees have been studied for over a decade. Several successful methods are based on association graphs where branch matchings are induced by

maximal cliques of a graph combining information from two trees (??); ? reported a labeling success rate of 97%. ? labeled airway branches recursively starting at the trachea as part of the segmentation process assigning labels from radius, orientation and bifurcation angle (90% success rate); this approach is vulnerable to differences in tree topology. ? matched tree paths rather than branches which avoids topological differences but discards the information stored in the tree structure and does not yield branch labels. ? matched airway branches using only branch shape without connectivity information (69% or 40% success rate depending on the features used).

In contrast to these methods which focus on either topology or branch geometry ? proposed a geodesic tree-shape model for anatomical labeling of airway branches that treats tree topology and branch geometry jointly. Each airway centerline tree is represented as a combinatorial tree whose edges carry landmark points sampled along the branches. Trees of different sizes and topologies are embedded in a common Euclidean space and different representations of the same shape are identified giving a quotient space equipped with the Quotient Euclidean Distance (QED). In this space any two trees are connected by a shortest deformation (geodesic) that can change the tree topology along the way. Since computing exact geodesics is NP-hard, the authors approximate them by limiting the number of topological changes in each lobar subtree. Branch correspondences obtained from the geodesics between an unlabeled tree and a set of expert-labeled trees are combined through majority voting to assign anatomical labels. ? later turned these geodesic distances into a hierarchical labeling scheme: a new airway tree is labeled from the root toward the periphery by matching it against a set of manually labeled training trees.

? proposed TopoField, a topology-aware implicit modeling framework for repairing incomplete pulmonary trees (airways, arteries, and veins) extracted from CT scans. The pulmonary tree is represented by sparse surface and skeleton point clouds which are fused through a surface-to-skeleton cross-attention module and projected onto a triplane implicit field that can be queried at arbitrary spatial locations. To avoid the need for annotated breakpoints the authors introduce TopoBreak a strategy that synthetically adds disconnections to already-incomplete trees and trains the network to recover them providing weak supervision for topology repair. On top of the shared implicit field, task-specific decoders jointly perform tree repair, anatomical labeling and lung segment reconstruction in a single forward pass.

III 2.3.4 Morphological and topological analysis of airway trees

Local airway geometry.

Early quantitative work focused on local airway geometry. ? related the wall area percent of a segmental bronchus and the extent of emphysema to lung function in smokers, and ? extended such measurements across airway generations using multiplanar reconstruction. ? used Pi10, a standardized measure of airway wall thickness, to show that wall thickening and emphysema aggregate independently in families. ? quantified the irregularity of the lumen radius along a single airway path and found it higher in COPD than in healthy subjects and asthma.

Global metrics of the tree.

Several global metrics have been derived from the segmented tree. ? introduced the total airway count (TAC) and showed that fewer visible branches predict COPD progression. ? characterized the complexity of the tree by its fractal dimension and linked lower values to morbidity and mortality in COPD, and later proposed the airway surface-area-to-volume ratio as a marker of airway remodeling associated with

airflow limitation (?). ? defined the airway-to-lung size ratio (dysanapsis) and showed that a smaller ratio is associated with COPD risk, while ? studied the physiological effects of this ratio, fractal dimension, and branch count in healthy never smokers.

Statistics on tree shapes.

? modeled anatomical trees as points in a space of tree-like shapes that captures both branch geometry and topological structure, and studied how to compute mean trees in this space, with airway trees as the application. ? represented airway trees as sets of attributed branches, matched the branches across subjects and classified COPD using a dissimilarity-based embedding with a k -NN classifier. ? proposed a family of geometric tree kernels for comparing anatomical trees whose branches carry vector-valued attributes such as branch shape, radius or airway wall area percentage. Unlike classical graph and tree kernels which rely on discrete node labels their path-based (all-pairs and root-path) and pointcloud kernels incorporate continuous geometric information while remaining efficient enough to scale to datasets of nearly 10,000 trees. Applying these kernels to airway trees segmented from low-dose CT scans of 1966 subjects they used kernel-based two-sample tests to show that airway trees of COPD patients and healthy individuals follow significantly different distributions. They also showed using SVM classification that geometry-aware kernels outperform average airway wall area percentage measures and state of the art graph kernels such as the shortest path and Weisfeiler - Lehman kernels in detecting COPD.

Topological descriptors.

? applied persistent homology to the airway tree using height as the filtration and showed that the resulting “upwards complexity” correlates with COPD severity. ? proposed two morphological metrics based on persistent homology to quantify tubular structures in chest CT images and applied them to the airway trees of patients with chronic obstructive pulmonary disease (COPD). They computed 0-dimensional persistent homology under two distance based filtrations. The first, the geodesic distance from the carina yields $treeH_0$, which describes the complexity of the branching structure. The second the distance from the lumen centerline yields $radialH_0$, which captures irregularities of the luminal surface in the central (1st to 5th generation) airways. On three cohorts acquired with different scanners and reconstruction kernels, both metrics were higher in COPD patients than in non-COPD smokers. In addition, $radialH_0$ predicted longitudinal FEV_1 decline independently of conventional CT measures such as emphysema extent (LAA%), airway-to-lung size ratio, and total airway count.

I 3 Datasets

[Optional short introduction: why these two datasets were used together, e.g. ATM'22 for the variety of sites and scanners, AIB23 for fibrotic lungs.]

II 3.1 The ATM'22 dataset

The Multi-site, Multi-domain Airway Tree Modeling dataset (ATM'22) was released as part of an official challenge held in conjunction with MICCAI 2022 (?). It targets the binary segmentation of the pulmonary airway tree from chest CT, and at the time of its release it was the largest publicly available airway dataset with complete voxel-wise annotations. The reference benchmark that preceded it, EXACT'09, provided

only 40 scans and never released its manual annotations, which had long limited the training and fair evaluation of data-driven methods.

The dataset contains 500 chest CT scans, partitioned into 300 training, 50 validation and 150 test cases. The test set is held privately by the organizers and is accessible only through the submission of a Docker container, which guarantees the fairness and reproducibility of the reported results. The scans originate from three sources, as summarized in Table ??.

Table 4: Composition of the ATM'22 dataset by split and by source site.

Split	Shanghai Chest Hospital	LIDC-IDRI	EXACT'09	Total
Training	226	54	20	300
Validation	20	30	–	50
Test	84 (58 COVID-19)	66	–	150
Total	330	150	20	500

For the 20 cases taken from EXACT'09, only the airway annotations are distributed; the corresponding CT volumes must be obtained separately from the original source.

The acquisition parameters are heterogeneous, which is the basis of the multi-site character of the dataset. The scans were acquired on Philips iCT 256, GE LightSpeed16 and Toshiba Aquilion scanners, the last of which appears only in the test set. Each slice is 512×512 pixels, and the number of slices per scan ranges from 157 to 1125, with a slice thickness between 0.45 and 1.00 mm and an in-plane resolution between 0.500 and 0.919 mm. The health status of the subjects ranges from healthy volunteers to patients with severe pulmonary disease. The multi-domain character comes from the 58 COVID-19 scans included in the test set: ground-glass opacity, consolidation and septal thickening alter the appearance of the lung parenchyma and blur the airway lumen, so these cases form a noisy domain on which the generalization ability of a model trained on clean scans can be assessed.

The annotations were produced in two stages. A preliminary segmentation was first obtained by ensembling, through majority voting, several airway segmentation models trained on the BAS dataset. These preliminary masks were then reviewed by three radiologists with more than five years of experience: two of them independently refined and cross-checked each scan, and the most senior radiologist validated the final annotation. The manual refinement required 60 to 90 minutes per scan and focused mainly on repairing breakages in the peripheral bronchi and removing small leakages into the surrounding parenchyma, using the original CT intensities and anatomical prior knowledge. The whole collection and annotation process took the organizers approximately one year.

The challenge evaluates submissions along two complementary axes. Topological completeness is measured by the tree length detected rate (TD) and the branch detected rate (BD),

$$TD = \frac{T_{det}}{T_{ref}}, \quad BD = \frac{B_{det}}{B_{ref}}, \quad (1)$$

where T_{det} and B_{det} denote respectively the total length and the number of branches correctly detected in the prediction, and T_{ref} and B_{ref} the corresponding quantities in the ground truth. Both metrics are computed on the largest connected component of the prediction, a branch being counted as detected when more than 80% of its centerline voxels lie within the ground truth. Topological correctness is measured by the Dice similarity coefficient (DSC) and the precision, with sensitivity and specificity reported as complementary indicators. The official ranking is based on the unweighted mean of TD, BD, DSC and precision. This dual evaluation reflects the fact that overlap-based measures alone are insufficient for tubular structures: a prediction may achieve a high Dice score while missing a substantial

number of distal branches, since the peripheral bronchi account for only a small fraction of the total airway volume.

3.2 The AIIB23 dataset

The AIIB23 dataset (Airway-Informed Quantitative CT Imaging Biomarker for Fibrotic Lung Disease 2023) was released as part of an official challenge held in conjunction with MICCAI 2023(?). It is the first publicly available dataset dedicated to airway segmentation in patients with fibrotic lung disease. Earlier public airway datasets, such as EXACT'09, BAS and ATM'22, mainly contain scans from healthy subjects or from patients with common pulmonary conditions such as nodules or COVID-19.

Fibrotic lung disease is particularly challenging for automatic airway segmentation for two reasons. First, honeycombing, the presence of small cystic airspaces in the lung periphery, visually resembles distal airway branches, so models often produce false-positive predictions known as airway leakages. Second, traction bronchiectasis and the distortion of lung tissue alter airway morphology, which frequently leads to missing or discontinuous branches. AIIB23 was therefore designed to benchmark segmentation methods on these difficult cases.

Data sources and composition.

The dataset gathers high-resolution computed tomography (HRCT) scans from three cohorts, acquired with different vendors and scanning protocols. The training and validation sets come from the Open Source Imaging Consortium (OSIC), an open-access database of patients with interstitial lung disease (ILD), idiopathic pulmonary fibrosis (IPF) and other respiratory diseases. The test set combines 90 fibrosis cases from the Australian IPF Repository (AIPFR) with 50 COVID-19 cases from the University Hospital of Parma. The COVID-19 cases make it possible to assess generalization to a different pathology. The patient populations are broadly comparable in age, with a median age of 68 years for OSIC, 70 for AIPFR and 65 for Parma. Table ?? summarizes the data splits.

Table 5: Composition of the AIIB23 dataset (?).

Split	Source	Task I (segmentation)	Task II (mortality)
Training	OSIC	120	95
Validation	OSIC	52	52
Test	AIPFR + Parma	140 ^a	90

^a90 fibrosis (AIPFR) and 50 COVID-19 (Parma) cases.

Image selection and preprocessing.

All scans were converted from DICOM to NIfTI format using SimpleITK. Only scans with more than 120 slices and an in-plane resolution of at least 512×512 pixels were retained. Despite this criterion, the number of slices varies considerably between cases. The fibrosis test scans contain significantly fewer slices than the training data, which results in lower image quality in the coronal and sagittal planes. This represents a domain shift between training and test data.

Annotation protocol.

The ground-truth airway masks were produced through a semi-automatic, multi-stage process. Preliminary labels were first generated by a deep learning model trained on the BAS dataset. Three junior radiologists then revised these labels, and two senior radiologists refined them by correcting breakages and mis-annotated voxels. When annotators disagreed, the most experienced radiologist's annotation was retained. Annotation was performed in ITK-SNAP using a drawing tablet. Inter-observer agreement on airway branches was very high (Cohen's $\kappa = 0.937$).

Evaluation.

The binary airway segmentations are evaluated with overlap metrics (intersection over union, IoU, and precision) and with topological metrics that are particularly relevant for thin tubular structures:

- the detected length ratio (DLR) is the fraction of the ground-truth branch length that the prediction recovers;
- the detected branch ratio (DBR) is the fraction of ground-truth branches that are correctly detected, a branch counting as detected when its IoU with the corresponding ground-truth branch is at least 0.8;
- the airway leakage ratio (ALR) quantifies false-positive predictions.

I 4 Registration

A common spatial reference was needed to explore and investigate how spatial alignment affects the learned shape representations and the subsequent unsupervised clustering, so we therefore first constructed a population template and then registered each airway tree to it.

II 4.1 Template construction

To obtain a common spatial reference for the pulmonary airway trees, a population-specific three-dimensional template was constructed from airway segmentation volumes using the Advanced Normalization Tools (ANTs) framework and its ANTsPy interface (?). Template construction is a *groupwise registration problem*: a common template and a set of transformations mapping the individual airway trees to it are estimated together. Unlike pairwise registration to an arbitrarily selected subject, groupwise registration does not designate one individual airway tree as the anatomical reference; the reference itself is progressively optimised to represent the population. The ANTs framework implements this as *symmetric groupwise normalization* (SyGN) (?). The registration of individual airway trees to the finished template is described separately in Section “??”.

III 4.1.1 Input data and image geometry

Selection of the input volumes.

Eight airway segmentation volumes were used: four drawn at random from ATM'22 and four from AIIB23. The volumes were passed to `ants.build_template` in the order ATM'22 first, then AIIB23. This

order matters for one implementation detail: the first volume defines the voxel grid of the template (see *Initialisation* in Section “??”).

Representation in physical space.

Each airway tree is a three-dimensional NIfTI image, a discrete voxel array together with spatial metadata that positions the array in physical space. A voxel is identified by a discrete index $\mathbf{i} = (i_x, i_y, i_z)^T$, whereas its anatomical location is a continuous physical coordinate $\mathbf{x} = (x, y, z)^T$, related by

$$\mathbf{x} = \mathbf{o} + \mathbf{D} \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & s_z \end{bmatrix} \mathbf{i}, \quad (2)$$

where $\mathbf{o}$ is the image origin, s_x, s_y, s_z are the voxel spacings and $\mathbf{D}$ is the direction matrix of the voxel axes, in NIfTI terminology these are encoded by the spatial affine in the image header. ANTs is built on ITK, in which registration is performed in *physical coordinates*, and ANTsPy converts the NIfTI geometry into the ITK representation of origin, spacing and direction. Differences in origin, orientation or voxel spacing between two airway segmentations are therefore taken into account when their structures are put into correspondence. The NIfTI affine defines the *initial coordinate system of each image*; it is not itself a registration.

An image I is treated as a function of physical position, and a transformation is applied by composition. If T is the fixed image (the template) and I the moving image, the warped image is defined on the domain of T :

$$(I \circ \Phi)(\mathbf{x}) = I(\Phi(\mathbf{x})), \quad \mathbf{x} \in \Omega_T. \quad (3)$$

Hence Φ maps each point of the *template* domain to its corresponding point in the *subject* domain, where the subject image is resampled. This is the convention used by ITK and ANTs; the inverse map Φ^{-1} takes subject points to template points.

III 4.1.2 Pairwise registration with SyN

Within every template-building iteration k , each airway segmentation I_i is registered to the current template $T^{(k)}$, which serves as the fixed image. The registration seeks $\Phi_i^{(k)}$ such that

$$I_i \circ \Phi_i^{(k)} \approx T^{(k)}, \quad (4)$$

i.e. after warping, corresponding airway structures occupy similar physical locations in template space. Inside `ants.build_template` this is the call

```
ants.registration(
    fixed=xavg, # current template T^(k)
    moving=image_i, # airway segmentation I_i
    type_of_transform='SyN'
)
```

The ANTsPy SyN preset.

Despite its name, the ANTsPy preset SyN contains three stages. The two images are first aligned by their centres of mass (a translation used as initial transform), an affine transformation $A_i^{(k)}$ is then estimated,

and finally the deformable symmetric normalisation $\phi_i^{(k)}$ is estimated between the template and the affinely transformed subject $I_i \circ A_i^{(k)}$. In [?](#), by contrast, “SyN” denotes only the deformable part, and rigid and scaling differences are assumed to have been removed beforehand. With the convention of Eq. [\(??\)](#), the total transformation is

$$\Phi_i^{(k)} = A_i^{(k)} \circ \phi_i^{(k)}, \quad I_i \circ \Phi_i^{(k)} = (I_i \circ A_i^{(k)}) \circ \phi_i^{(k)}, \quad (5)$$

so a template point is moved first by the deformable map and then by the affine map into subject space. The affine part accounts for global translation, rotation, scaling and shear; the deformable part accounts for spatially varying differences that no single affine map can represent. (The centre-of-mass translation is folded into the affine stage.)

Registration parameters.

Apart from `type_of_transform`, no argument was specified, so all other settings were ANTsPy defaults: Mattes mutual information as the similarity metric of both the affine and the deformable stage; an affine stage with (2100, 1200, 1200, 10) iterations on four resolution levels; a deformable stage with (40, 20, 0) iterations from coarse to fine; an optimiser step `grad_step` = 0.2; and Gaussian smoothing of the update field (`flow_sigma` = 3) and of the total field (`total_sigma` = 0). Because the last entry of the deformable iterations is zero, no deformable optimisation is performed at full image resolution, which limits how precisely thin peripheral airway branches can be aligned. The SyN step `grad_step` is unrelated to the template shape-update step `gradient_step` of Section [“??”](#), although both equal 0.2.

Similarity metric.

Mutual information (MI) measures the statistical dependence between the intensity distributions of the fixed and warped moving image, estimated from their joint histogram. It is *not* the metric used in either reference publication. [?](#) developed SyN with *local cross-correlation* (CC), arguing that MI is a global measure well suited to rigid registration but potentially limiting for deformable registration, and that a localised MI estimate becomes unreliable because joint probabilities need many samples; local CC depends only on local means and variances, which can be estimated from few samples. SyGN ([?](#)) likewise uses local CC both for the registrations and for the template appearance. With the mean-subtracted images $\bar{I} = I - \mu_I$ and $\bar{J} = J - \mu_J$, the means being computed in a small cubic neighbourhood of $\mathbf{x}$, local CC is

$$CC(\bar{I}, \bar{J}, \mathbf{x}) = \frac{\langle \bar{I}, \bar{J} \rangle^2}{\langle \bar{I}, \bar{I} \rangle \langle \bar{J}, \bar{J} \rangle}, \quad (6)$$

with inner products over the same neighbourhood (5^3 voxels in [?](#), radius 4 in [?](#)). Because the airway volumes are segmentations, any metric compares the spatial patterns of the label values rather than anatomical intensities.

Registration as optimisation.

Deformable registration is often written as

$$\hat{\Phi} = \arg \min_{\Phi} D(T, I \circ \Phi) + \lambda R(\Phi), \quad (7)$$

with a dissimilarity term D , a regulariser R and a weight λ . This is the classical elastic formulation. SyN departs from it: [?](#) restrict the search to diffeomorphisms and optimise the similarity term essentially

without a competing penalty. Regularity comes from the diffeomorphic path structure and from smoothing of the velocity field; the regularisation term contributes little to the total energy but ensures that a path of minimal length is found.

Diffeomorphisms generated by velocity fields.

A diffeomorphism is a differentiable map with a differentiable inverse. In SyN, as in other large-deformation methods, it is generated by integrating a time-dependent velocity field $\mathbf{v}(\mathbf{x}, t)$:

$$\frac{\partial \phi(\mathbf{x}, t)}{\partial t} = \mathbf{v}(\phi(\mathbf{x}, t), t), \quad \phi(\mathbf{x}, 0) = \mathbf{x}. \quad (8)$$

The regularity of $\mathbf{v}$ is measured by a norm $\|\mathbf{v}\|_L$ induced by a linear differential operator such as $L = a\nabla^2 + b\text{Id}$, and the path length

$$D(\phi(\cdot, 0), \phi(\cdot, 1)) = \int_0^1 \|\mathbf{v}(\cdot, t)\|_L dt \quad (9)$$

defines a distance between shapes; its infimum over admissible velocity fields is a true metric, whose shortest paths are geodesics. This distance is what SyGN later minimises to find the average shape (Section “??”).

Symmetric two-part formulation.

SyN splits the path between two images I and J into two halves. Two diffeomorphisms, ϕ_1 acting on I and ϕ_2 acting on J , are generated by Eq. (??) with their own velocity fields, each integrated from the identity only over $t \in [0, 0.5]$. The two images are deformed towards each other and compared at the midpoint (figure ??):

$$E_S(I, J) = \inf_{\phi_1} \inf_{\phi_2} \int_0^{0.5} \left(\|\mathbf{v}_1(\cdot, t)\|_L^2 + \|\mathbf{v}_2(\cdot, t)\|_L^2 \right) dt + \int_{\Omega} \Pi(I \circ \phi_1(\cdot, 0.5), J \circ \phi_2(\cdot, 0.5)) d\Omega, \quad (10)$$

where Π is a pointwise dissimilarity (the squared intensity difference in the derivation of ?, $-CC$ in their main method). The two half-paths are kept at equal length, so both images travel the same distance and meet at the *mean shape* between them; SyN thus solves a two-image mean problem, the pairwise counterpart of the template estimation in Section “??”.

Full transformations by composition.

The complete mapping between the two images is obtained by joining the half-paths:

$$\phi_1(\mathbf{x}, 1) = \phi_2^{-1}(\phi_1(\mathbf{x}, 0.5), 0.5), \quad \phi_1^{-1}(\mathbf{x}, 1) = \phi_1^{-1}(\phi_2(\mathbf{x}, 0.5), 0.5). \quad (11)$$

Because I and J enter Eq. (??) in the same way, exchanging them yields the same path in the opposite direction: the result does not depend on which image is called “fixed” and which “moving”.

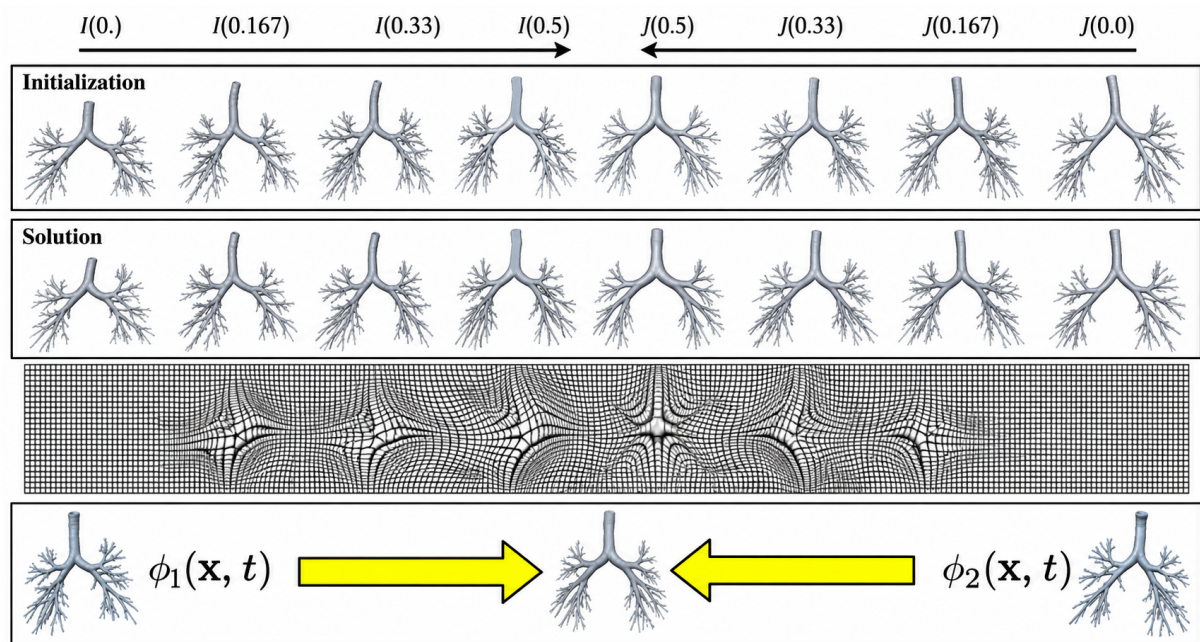


Figure 6: Schematic of the symmetric SyN path between two airway trees I (left) and J (right). Top: the two images at initialisation. Middle: after convergence, both images are deformed along ϕ_1 and ϕ_2 and meet at the mean shape at $t = 0.5$. The full map from I to J is obtained by joining the two half-paths (Eq. ??). Illustrative only; not computed from the study data.

Explicit, accurate inverses.

Diffeomorphisms are invertible in the continuous setting, but interpolation errors in a discrete implementation accumulate and can break invertibility. SyN therefore computes the inverses ϕ_i^{-1} explicitly with a fixed-point algorithm and enforces $\phi_i^{-1}(\phi_i) = \text{Id}$ and $\phi_i(\phi_i^{-1}) = \text{Id}$ to sub-voxel accuracy. This distinguishes SyN from inverse-consistent registration, in which forward and backward maps are estimated separately and made consistent only approximately through penalty terms. Both the subject-to-template and the template-to-subject mappings are therefore available and consistent.

Optimisation in practice.

Each SyN iteration deforms both images to the midpoint, computes the gradient of the similarity term with respect to each half-map, smooths it into velocity updates, updates ϕ_1 and ϕ_2 through a discretisation of Eq. (??) with a sub-voxel step, recomputes the inverses and forms the maps of Eq. (??). The operator L is approximated by a Gaussian kernel applied to the *incremental* update rather than to the total displacement (flow_sigma and total_sigma in ANTsPy); smoothing the increment is what permits large diffeomorphic deformations, whereas smoothing the total displacement, as in elastic and Demons-type methods, restricts the solution to small deformations. The optimisation runs from coarse to fine resolution levels.

4.1.3 Groupwise template construction with SyGN

Formulation.

SyGN seeks the template and the set of transformations that give the “smallest” parameterisation of the dataset, measured by the SyN energy of Eq. (??). The template is described by an appearance $\bar{I}$ (its intensities) and a shape, represented by a diffeomorphism ψ that defines the template’s coordinate system. Given the images $\{J^i\}_{i=1}^N$, ? minimise

$$E_{\bar{I}} = \sum_{i=1}^N E_S(\bar{I}, J^i, \phi^i) \quad \text{with} \quad \phi^i(\mathbf{x}, 0) = \psi(\mathbf{x}) \quad \forall i, \quad (12)$$

jointly over the mappings ϕ^i , the appearance $\bar{I}$ and the shape ψ . No initial template has to be chosen by the user, and the stated goals are that the final template’s appearance does not depend on any specific anatomy and its shape does not depend on any individual’s coordinate system.

Coordinate-descent algorithm.

Eq. (??) is minimised by coordinate descent, one group of variables at a time with the others fixed. After initialisation, each iteration (1) registers every image to the fixed template with SyN; (2) updates the template appearance $\bar{I}$; (3) updates the template shape ψ towards the average of the mappings; and (4) resamples the template through the new ψ , resets the mappings to start from it, and repeats. The template is only updated after all registrations of an iteration are complete, so appearance and shape are never estimated from partially optimised mappings. `ants.build_template` implements the same loop, but with a different initialisation and a different appearance update, as described next (figure ??).

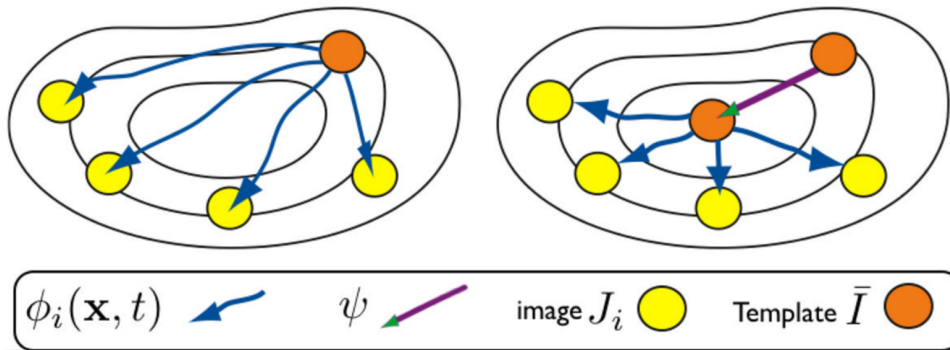


Figure 7: Two steps of one SyGN iteration. Left: the diffeomorphic paths ϕ_i from the current template $\bar{I}$ (orange) to the images J_i (yellow) are estimated (Step 1). Right: the template shape is moved by ψ towards the population centre, which shortens the total path length (Step 3). Reproduced from ?, Fig. 2.

Initialisation.

In ?, the first template is the voxelwise (Euclidean) average of the images *after affine alignment*, and all ϕ^i and ψ start as the identity. In the present implementation no initial template was supplied, and ANTsPy forms the initial template *without any alignment*: each image is multiplied by its weight $w_i = 1/N$,

resampled onto the voxel grid of the first input image I_1 , and summed,

$$T^{(0)}(\mathbf{x}) = \sum_{i=1}^N w_i I_i(\mathbf{x}), \quad \mathbf{x} \in \Omega_{I_1}. \quad (13)$$

Two consequences follow. First, $T^{(0)}$ averages the airway trees as they lie in their own scanner coordinates, so it is blurred and its quality depends on how well these coordinate systems already agree. Because the inputs are binary segmentations, it also contains intermediate values equal to the fraction of volumes that contain airway at each location. Second, every later template is resampled on this grid, so the origin, spacing, orientation and field of view of the final template are inherited from I_1 , here the first randomly selected ATM'22 volume. The template's *shape* is still optimised over the whole population, but its *grid* is not independent of the input order.

Step 1 – registration.

With $T^{(k)}$ fixed, each I_i is registered to it with the ANTsPy SyN preset (Section “??”), giving the affine maps $A_i^{(k)}$, the deformable maps $\phi_i^{(k)}$ and the warped images $I_i \circ \Phi_i^{(k)}$, all on the template grid.

Step 2 – appearance update.

?? do not take the template appearance to be a plain average. They optimise it by gradient ascent on the cross-correlation of Eq. (??) between the template and the warped images (intensities normalised to $[0, 1]$, gradients smoothed with a Gaussian of $\sigma = 1$ voxel, step size 0.1), and show that this preserves more contrast and detail than the Euclidean mean. ANTsPy does *not* implement this step. Its appearance is the weighted Euclidean mean of the warped images,

$$T_{\text{avg}}^{(k+1)}(\mathbf{x}) = \sum_{i=1}^N w_i \left[I_i \circ \Phi_i^{(k)} \right](\mathbf{x}), \quad (14)$$

and, at the end of each iteration (after the shape update below), this image is blended with a sharpened copy of itself,

$$T^{(k+1)} = \beta T_{\text{shape}}^{(k+1)} + (1 - \beta) \text{Sharpen}(T_{\text{shape}}^{(k+1)}), \quad (15)$$

with $\beta = \text{blending_weight} = 0.75$. The sharpening blend is a heuristic of the implementation that compensates for the blurring of the mean; it appears in neither publication and does not deform the template. In both the paper and the implementation, averaging happens *after* spatial alignment, so each template location combines corresponding locations of the airway trees rather than identical voxel indices of the original images.

Step 3 – shape update.

Averaging aligned images fixes the appearance but not the *position* of the template within the population: if the current template is closer to some subjects than to others, the mappings to those subjects are short and the others long (figure ??, right). ? therefore move the template towards the Fréchet (intrinsic) mean of the diffeomorphisms, the shape ψ that minimises the total squared geodesic distance $\sum_i D^2(\psi, \phi^i(\cdot, 1))$ of Eq. (??). Velocity fields, unlike diffeomorphisms, can be added, so the average

can be taken over the initial velocity fields of the mappings; setting the derivative of the total distance to zero gives the optimality condition

$$\bar{\mathbf{v}} = \sum_{i=1}^N \mathbf{v}^i(\psi, 0) = 0, \quad (16)$$

and, away from the optimum, the template shape evolves along $d\psi/d\bar{t} = \bar{\mathbf{v}}(\psi, \bar{t})$. Under a small-deformation approximation, with $\psi = \text{Id}$ at the current template, one step of length $\Delta\bar{t}$ becomes a scaled sum of the displacements of the mappings:

$$\psi_{\Delta\bar{t}}(\mathbf{x}) = \mathbf{x} + \frac{\Delta\bar{t}}{\Delta t} \sum_{i=1}^N (\phi_1^i(\mathbf{x}, \Delta t) - \mathbf{x}). \quad (17)$$

At the average configuration the displacements cancel and the template no longer moves. ANTsPy implements this approximation: it averages the forward displacement fields $W_i^{(k)}$ of the deformable registrations, $W^{(k)} = \sum_i w_i W_i^{(k)}$, scales the result by $-\alpha$ with $\alpha = \text{gradient_step} = 0.2$ (the counterpart of $\Delta\bar{t}$), combines it with the inverse of the average affine transformation $\bar{A}^{(k)}$ of the registrations, and resamples the averaged image through the resulting map $\Psi^{(k)}$:

$$T_{\text{shape}}^{(k+1)} = T_{\text{avg}}^{(k+1)} \circ \Psi^{(k)}, \quad \Psi^{(k)} \text{ built from } -\alpha W^{(k)} \text{ and } (\bar{A}^{(k)})^{-1}. \quad (18)$$

Two simplifications relative to the paper should be noted: ANTsPy averages total displacement fields rather than initial velocity fields, and it also re-centres the template affinely. With `useNoRigid=True` (the default), the rigid part is excluded from $\bar{A}^{(k)}$, so only the average scaling and shear are removed and the template does not drift by an arbitrary common rotation or translation. Eq. (??) is schematic; the exact composition order follows the ANTsPy source.

Why the shape update matters for segmentation volumes.

? illustrate this step (Figure ??) with a synthetic set of four binary ellipses with the same centre and different radii. The initial average has intermediate grey levels, like the averaged airway masks here. With the shape update, SyGN converges, up to interpolation error, to the true mean shape. Without it, optimising appearance alone converges to a geometrically wrong template, the 0.5 level set of the initial average. The shape update is therefore what makes the geometry of a template built from binary segmentations reflect the population average rather than the initialisation; setting `gradient_step > 0` is essential for this.

Iterations and convergence.

The loop was run for a fixed number of 8 iterations (the ANTsPy default is 3). ? report that SyGN typically converges in fewer than ten iterations, usually three to five, depending on the complexity of the deformations. No stopping criterion is applied by the code. However, ANTsPy prints the mean absolute value of the average displacement field $W^{(k)}$ at every iteration; by Eq. (??) this quantity should tend towards zero as the template approaches the population mean, and it can be used to assess convergence.

Unbiasedness and its limits.

In principle SyGN gives a template that is unbiased: it requires no user-selected reference, uses a symmetric pairwise method, and centres the template shape on the population. In the present

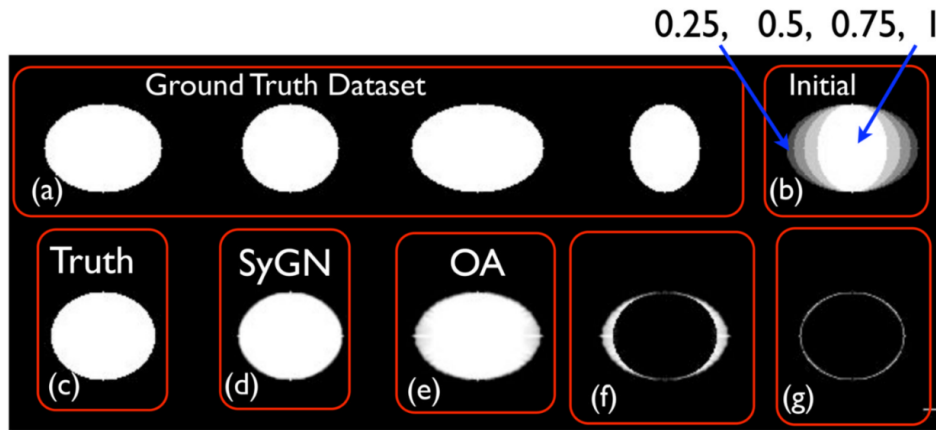


Figure 8: Effect of the shape update on binary images. (a) Four binary ellipses; (b) their initial average, with grey levels 0.25–1; (c) true mean shape; (d) SyGN result with shape update; (e) result without shape update (appearance only), which converges to the wrong shape; (f,g) errors of (e) and (d) with respect to (c). Reproduced from [?], Fig. 3.

implementation this holds for the template shape up to three qualifications: the template grid is inherited from the first input volume (Eq. (??)); the eight volumes are a small random subset of the two datasets; and the registrations and the appearance use MI and a sharpened mean instead of the cross-correlation model of the reference publications.

Output.

After the eighth iteration, the template $T^{(8)}$ was written to `template.nii.gz`. It is the reference used in Section “?”. The complete procedure is summarised in Figure ??

The NIfTI geometry defines where each original voxel grid lies in physical space, whereas the affine and diffeomorphic transformations estimated by ANTs establish anatomical correspondence between these physical domains. The final template is therefore a population-derived coordinate space in which the anatomical variability of the eight airway trees has been brought into correspondence.

4.2 Affine registration

Airway trees extracted from different patients differ in position, orientation and overall size, as a consequence of patient positioning in the scanner, differences in body size and acquisition parameters. To make them directly comparable, all airway segmentations were mapped into the constructed template. An affine transformation was chosen for this purpose because it compensates for these global differences while preserving the local geometry of each airway tree, which is the object of the subsequent analysis.

Affine registration consists of finding the spatial transformation T that best aligns a *moving* image M with a *fixed* image F . In this work, the fixed image is the template and each AIB23 segmentation is, in turn, the moving image. The transformation is restricted to the class of three-dimensional affine maps,

$$T_{\mu}(\mathbf{x}) = \mathbf{A}(\mathbf{x} - \mathbf{c}) + \mathbf{c} + \mathbf{t}, \quad (19)$$

where $\mathbf{x} \in \mathbb{R}^3$ is a point in the fixed image domain, $\mathbf{A} \in \mathbb{R}^{3 \times 3}$ is a general matrix, $\mathbf{t} \in \mathbb{R}^3$ is a translation vector and $\mathbf{c}$ is a fixed centre of rotation. The parameter vector μ therefore contains $9 + 3 = 12$

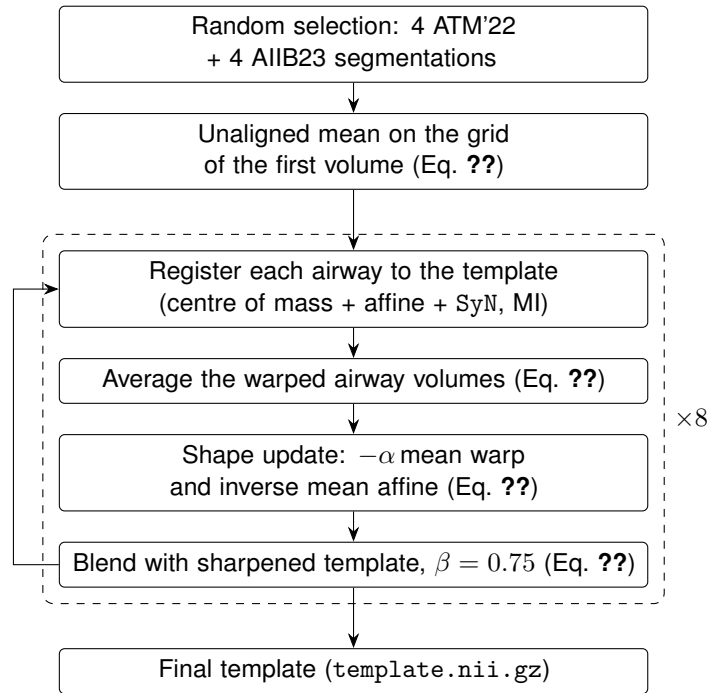


Figure 9: Groupwise template construction with `ants.build_template`. The dashed block is one SyGN iteration, repeated eight times.

degrees of freedom, which jointly encode translation, rotation, anisotropic scaling and shear. Because the transformation is linear up to a translation, straight lines are mapped to straight lines and parallelism is preserved, so the shape of the airway tree is modified only globally.

Similarity metric and optimisation.

The optimal parameters are obtained by solving

$$\hat{\mu} = \arg \min_{\mu} \mathcal{C}(F, M \circ T_{\mu}), \quad (20)$$

where $\mathcal{C}$ is a cost function measuring the dissimilarity between the fixed image and the transformed moving image. The cost function used is the negative of the Mattes mutual information (?),

$$\mathcal{C}(\mu) = - \sum_{\iota} \sum_{\kappa} p(\iota, \kappa; \mu) \log \frac{p(\iota, \kappa; \mu)}{p_F(\iota) p_M(\kappa; \mu)}, \quad (21)$$

where $p(\iota, \kappa; \mu)$ is the joint probability of intensity bins ι and κ in the fixed and transformed moving images, and p_F and p_M are the corresponding marginal distributions. In the Mattes formulation, these distributions are estimated with B-spline Parzen windows, which makes the metric differentiable with respect to μ . The histograms were built with 32 bins from a random sample of 20% of the voxels, which reduces the computational cost of each iteration.

Equation (??) was solved with a gradient descent optimiser. Before optimisation, the transformation was initialised by aligning the intensity centres of mass of the two images, which provides a coarse translational alignment and reduces the risk of convergence to a poor local optimum.

Multi-resolution strategy.

To improve robustness and speed, the optimisation was performed within a four-level multi-resolution scheme (?). At each level, both images were smoothed with a Gaussian kernel and downsampled by an integer shrink factor; the optimisation then proceeded from the coarsest level to the original resolution, with the result of each level used to initialise the next. Coarse levels capture the large-scale alignment, while finer levels refine it. The settings of each level are listed in Table ??.

Table 6: Multi-resolution settings of the affine registration.

Level	Shrink factor	Gaussian σ (voxels)	Maximum iterations
1 (coarsest)	6	3	2100
2	4	2	1200
3	2	1	1200
4 (finest)	1	0	10

At each level, the optimisation stopped either when the maximum number of iterations was reached or when the cost function had converged, i.e. when its variation over the last 10 iterations fell below 10^{-6} .

Resampling of the registered images.

Once the optimal transformation $T_{\hat{\mu}}$ had been estimated, each moving segmentation was resampled onto the voxel grid of the template using linear interpolation and saved in NIfTI format. Since the inputs are binary masks, linear interpolation yields intermediate values in $[0, 1]$ at the boundary of the airway lumen, while voxels well inside or outside the airway keep the values 1 and 0 respectively.

I 5 Encoding

[Optional short introduction: why a learned representation of the airway trees is needed, and the three approaches tried – a variational autoencoder, deterministic and sparse autoencoders, and an implicit encoding.]

II 5.1 Variational Autoencoder for Airway Tree Representation Learning

To learn a compact representation of airway trees we first trained a Variational Autoencoder (VAE) on the the binary voxel occupancy grid. As ? point out there is no meaningful way to interpolate directly between two binary grids. Relating voxelized shapes to one another therefore requires a model that works in an abstract feature space capturing the main factors of variation. A VAE is a probabilistic framework that learns two networks jointly:

- an **inference (encoder) network**, which maps an input to a set of latent variables describing it, and
- a **generative (decoder) network**, which maps from the latent space back to the input space.

Once the encoder has learned latent variables that describe the underlying variation between shapes each airway tree can be encoded as a latent vector. That vector can be used as a learned representation

or interpolated and decoded back into a voxel grid. Our models are adapted from the voxel-based VAE of ?.

III 5.1.1 Model architecture

Each airway tree was sownsampled at two resolutions, $128 \times 128 \times 128$ and $256 \times 256 \times 256$ voxels. The higher resolution preserves more of the thin distal branches at the cost of a much larger memory and computational footprint. We trained a separate VAE for each resolution. Both networks follow the design principles of ? adapted to the larger input size. The 256^3 model has one additional downsampling stage (two extra encoder layers), so that both networks reach the same $7 \times 7 \times 7$ spatial grid at the bottleneck. The architectures are summarized in Figure ?? and Table ??.

Encoder.

The encoder is a stack of 3D convolutional blocks. Each block consists of a $3 \times 3 \times 3$ convolution without bias, instance normalization with learnable affine parameters, and an ELU activation. As in ?, downsampling is performed with strided convolutions rather than pooling, in every second layer. The layers alternate between an unpadded stride-1 convolution, which reduces each spatial dimension by 2, and a stride-2 convolution with padding 1, which approximately halves it. The number of filters doubles at every layer:

- **128^3 model:** 8 convolutional layers, starting with 2 filters and reaching 256 filters. The spatial size goes $128 \rightarrow 126 \rightarrow 63 \rightarrow 61 \rightarrow 31 \rightarrow 29 \rightarrow 15 \rightarrow 13 \rightarrow 7$.
- **256^3 model:** 10 convolutional layers, starting with 4 filters and reaching 2048 filters. The spatial size goes $256 \rightarrow 254 \rightarrow 127 \rightarrow 125 \rightarrow 63 \rightarrow 61 \rightarrow 31 \rightarrow 29 \rightarrow 15 \rightarrow 13 \rightarrow 7$.

A $1 \times 1 \times 1$ convolution followed by instance normalization and ELU then reduces the number of channels to 32 while keeping the 7^3 grid. The result is flattened into a vector of $32 \cdot 7^3 = 10,976$ features. The flattened features are then mapped directly by two separate linear layers to the mean μ and the log standard deviation $\log \sigma$ of the latent Gaussian. The log standard deviation is clamped to $[-10, 10]$ for numerical stability. During training a latent vector is sampled with the reparameterization trick

$$\mathbf{z} = \mu + \exp(\log \sigma) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \quad (22)$$

At inference time the mean is used directly ($\mathbf{z} = \mu$), giving a deterministic encoding of each airway tree.

Decoder.

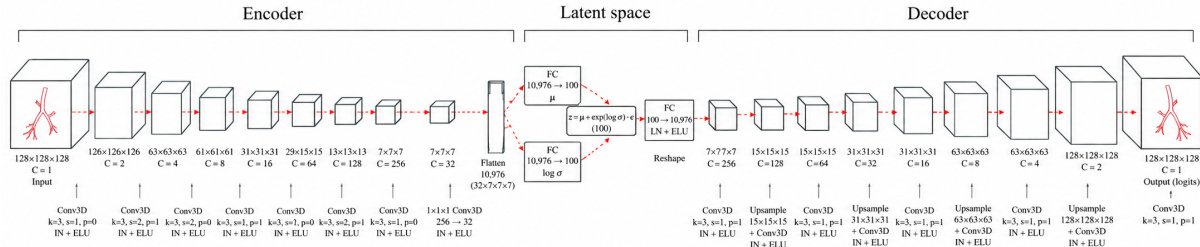
The decoder mirrors the encoder, and its weights are not tied to the encoder's. A linear layer maps $\mathbf{z}$ back to 10,976 features, followed by layer normalization (?) and ELU, and the result is reshaped into a 32×7^3 volume. A stride-1 convolutional block expands the channels to the width of the deepest encoder layer (256 for the 128^3 model, 2048 for the 256^3 model). The decoder then alternates between upsampling blocks and stride-1 convolutional blocks (padding 1, spatial size preserved), halving the number of channels at every layer. Finally, we use a resize-convolution, each upsampling block applies nearest-neighbour upsampling to an exact target size, followed by a $3 \times 3 \times 3$ convolution, instance normalization and ELU. This avoids the checkerboard artifacts associated with transposed convolutions (?) and lets the decoder match the odd intermediate sizes of the encoder exactly. The spatial size goes

$7 \rightarrow 15 \rightarrow 31 \rightarrow 63 \rightarrow 128$ for the 128^3 model and $7 \rightarrow 15 \rightarrow 31 \rightarrow 63 \rightarrow 127 \rightarrow 256$ for the 256^3 model. A final $3 \times 3 \times 3$ convolution with bias maps the features to a single output channel with no normalization or activation. Its outputs are logits the sigmoid applied inside the loss converts them into the predicted probability that each voxel is occupied.

All layers use the ELU activation (?), except the output layer. Where Brock et al. use Batch Normalization (?), we use instance normalization (?). At 128^3 and 256^3 resolution, GPU memory limits the batch size to a few samples, which makes batch statistics unreliable; instance normalization normalizes each sample independently and does not depend on batch size. Convolutional and linear weights are initialized with Glorot (Xavier) normal initialization (?), and biases are initialized to zero.

Table 7: Summary of the two VAE architectures.

Stage	128^3 model	256^3 model
Input	1×128^3	1×256^3
Encoder conv layers	8	10
Filters (first $\rightarrow$ last)	$2 \rightarrow 256$	$4 \rightarrow 2048$
Encoder output	256×7^3	2048×7^3
$1 \times 1 \times 1$ reduction	32×7^3	32×7^3
Flattened features	10,976	10,976
Latent dimension	256	256
Decoder upsampling steps	4	5
Decoder filters (first $\rightarrow$ last)	$256 \rightarrow 2$	$2048 \rightarrow 4$
Output	1×128^3 (logits)	1×256^3 (logits)

Figure 10: Architecture of the voxel-based VAE adapted from ?. Shown for the 128^3 model.

5.1.2 Loss function

Following ?, the loss combines three terms, the KL divergence between the approximate posterior and the prior on the latent variables, L2 weight regularization and a reconstruction term based on a modified Binary Cross-Entropy (BCE). We then add a fourth term, cIDice, described below.

Modified BCE.

The standard BCE is

$$\mathcal{L}_{\text{BCE}} = -t \log(o) - (1 - t) \log(1 - o), \quad (23)$$

where $t \in \{0, 1\}$ is the target voxel value and $o \in (0, 1)$ is the network output. ? identify two problems with this loss for voxel data. First, its derivative with respect to o becomes very small as o approaches t which can cause vanishing gradients. Second, it penalizes false positives and false negatives equally. Most of a voxel grid is empty so the network can fall into a poor local optimum by predicting every voxel

as empty. The second problem is even more severe for airway trees than for the ModelNet objects in the original paper, because airways are thin, branching structures that occupy a very small fraction of the grid.

¶ propose two modifications to address these problems: shifting the target and output ranges to increase the gradient magnitude and weighting false negatives against false positives. In our implementation we adopt the second modification. The targets remain binary $t \in \{0, 1\}$ and the output o is the sigmoid of the decoder logits clamped to $[\epsilon, 1 - \epsilon]$ with $\epsilon = 10^{-7}$ to avoid $\log(0)$. A hyperparameter γ weights false negatives against false positives:

$$\mathcal{L}_{\text{BCE-mod}} = -\gamma t \log(o) - (1 - \gamma)(1 - t) \log(1 - o). \quad (24)$$

The loss is summed over all voxels of a sample and averaged over the batch. We set $\gamma = 0.99$, which strongly penalizes missed voxels and reduces the penalty for spurious ones, this is slightly higher than the 0.97 used by Brock et al., reflecting the even stronger class imbalance of airway trees. The KL divergence is summed over the latent dimensions and averaged over the batch, and it is weighted by a factor β , following the β -VAE formulation (¶). L2 weight regularization is applied through the optimizer's weight decay.

clDice

Voxel-wise losses do not explicitly preserve the connectivity of tubular structures which is essential for airway trees. We therefore add the centerline Dice (clDice) loss (¶) to the reconstruction objective. Let V_P and V_L be the predicted and ground-truth volumes, and S_P and S_L their soft skeletons. clDice is defined as

$$T_{\text{prec}} = \frac{|S_P \cap V_L|}{|S_P|}, \quad T_{\text{sens}} = \frac{|S_L \cap V_P|}{|S_L|}, \quad (25)$$

$$\text{clDice} = 2 \frac{T_{\text{prec}} \cdot T_{\text{sens}}}{T_{\text{prec}} + T_{\text{sens}}}. \quad (26)$$

Topology precision is the fraction of the predicted skeleton lying inside the ground-truth volume, and topology sensitivity is the fraction of the ground-truth skeleton lying inside the predicted volume. The final training objective is

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{BCE-mod}} + \beta \mathcal{L}_{\text{KL}} + \lambda (1 - \text{clDice}) + \mathcal{L}_{\text{L2}}, \quad (27)$$

III 5.1.3 Training

The model is trained with stochastic gradient descent using Nesterov momentum (¶). Training runs until the reconstruction error on a held-out validation set stops decreasing. Following ¶, the learning rate is 0.0001 for the first epoch and then raised to 0.001. The dataset contains 419 airway trees, which were split into training, validation and test sets using an 80/10/10 split, resulting in 335 training, 42 validation and 42 test samples.

III 5.1.4 Results

During training, the reconstruction loss on the training set decreased steadily, and the model reconstructed training airway trees well. Performance on held-out data was much worse. The validation loss did not follow the training loss (Figure ??). Reconstructions of validation and test trees were poor, missing branches, broken connectivity, blurred or blob-like structures (Figure ??).

This gap between training and validation performance shows that the model **overfits**, it memorizes the training samples instead of learning a representation that generalizes to unseen airway trees.

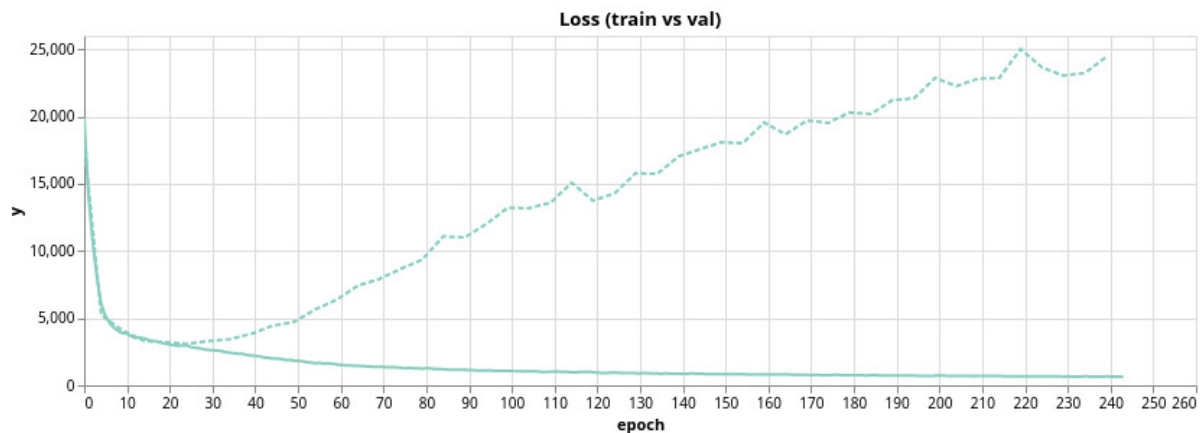


Figure 11: Training and validation losses of the VAE. The validation loss does not follow the decrease of the training loss, indicating overfitting.

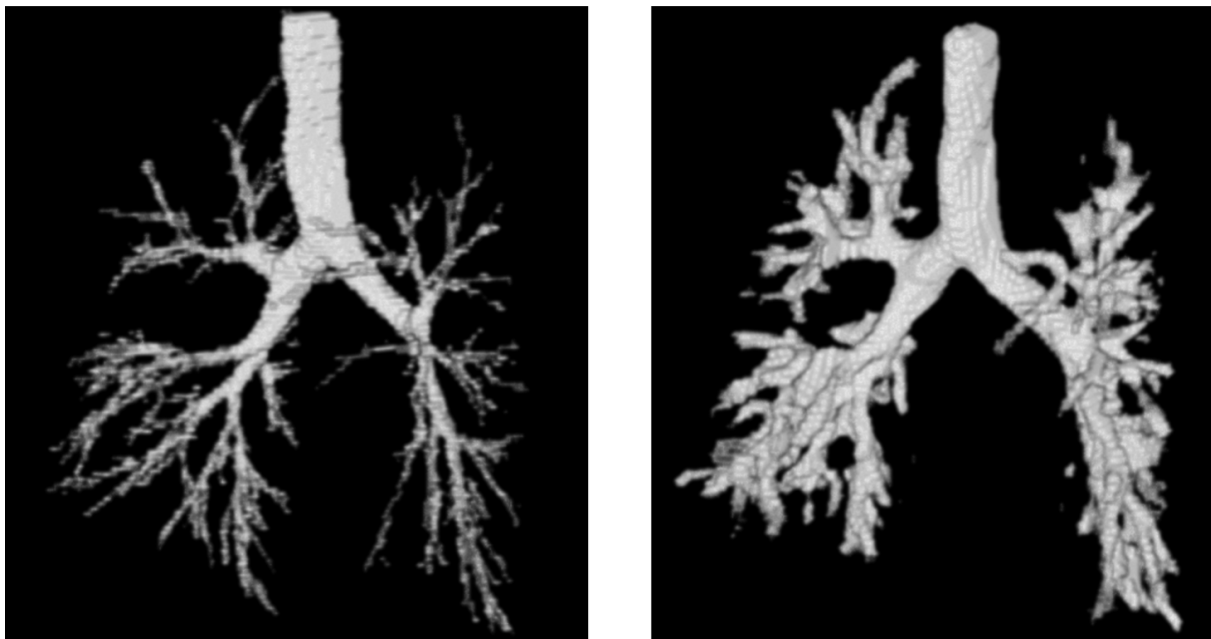


Figure 12: Original airway tree and their VAE reconstructions for training (left) and validation (right).

III 5.1.5 Mitigating overfitting

To reduce overfitting, we considered data augmentation and weight regularization. Each training sample could be augmented by a random rotation about the cranio-caudal axis, with the rotation angle sampled uniformly from -30° to 30° . These augmented copies increased the variability of the training data while preserving the overall airway topology. In addition, L_2 weight regularization was applied through the weight decay term of the AdamW optimizer. Dropout layers with rate $p = 0.2$ were added to the fully connected layers to discourage co-adaptation of features and reduce overfitting.

Neither strategy resolved the problem. As shown in Figure ??, the training and validation curves still

diverge after adding augmentation and dropout, and validation and test reconstructions remain poor. This suggests the limitation lies deeper than a lack of regularization. Plausible causes include the small size of the dataset relative to model capacity, the low voxel resolution relative to the thinness of the airway branches, and the difficulty voxel VAEs have with thin structures.

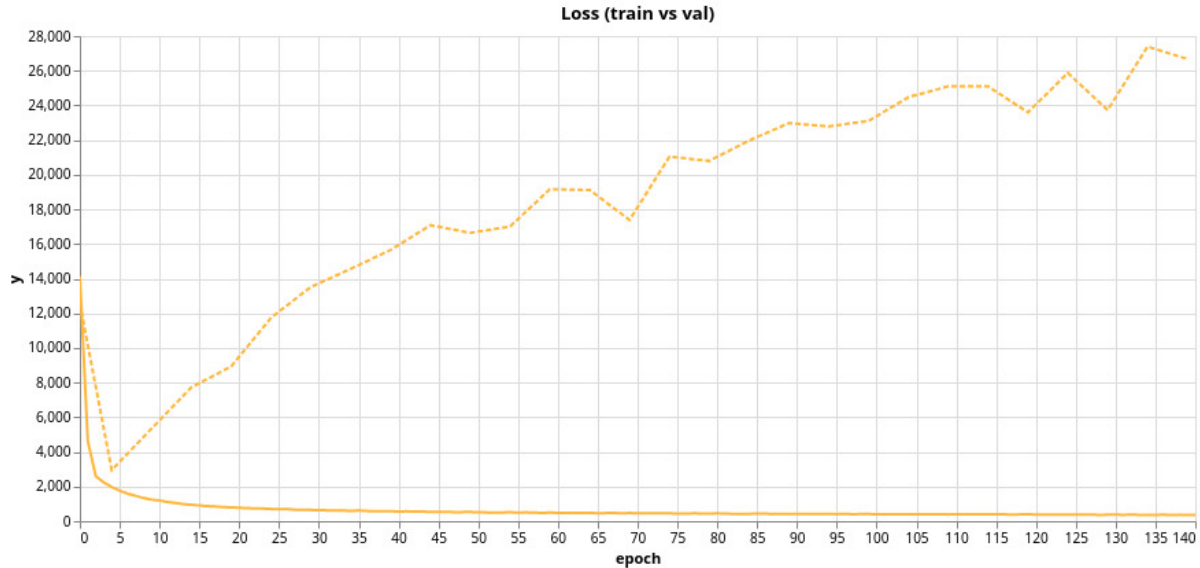


Figure 13: Training and validation losses after adding data augmentation and dropout, the gap between training and validation performance persists.

II 5.2 Deterministic and Sparse Autoencoders

In addition to the VAEs, we trained two further variants at both input resolutions 128^3 and 256^3 . The first is a deterministic autoencoder and the second is a model with a sparse convolutional encoder.

III 5.2.1 Deterministic autoencoder

The deterministic autoencoder uses the same encoder and decoder as the VAE with the variational part removed. Instead of two linear layers predicting μ and $\log \sigma$, a single linear layer maps the flattened encoder features directly to a latent code z of dimension 256. No sampling takes place and the KL divergence term is dropped from the loss so the objective reduces to the weighted BCE, the cDice term and L2 weight regularization:

$$\mathcal{L}_{AE} = \mathcal{L}_{BCE-mod} + \lambda (1 - cDice) + \mathcal{L}_{L2}. \quad (28)$$

Without the KL term, the latent space is no longer pushed towards a unit Gaussian prior. This means the model cannot be used to generate new trees by sampling, but the latent code can still serve as a learned representation of each airway tree. Removing the KL term also removes the trade-off between reconstruction quality and latent regularization. This lets us test whether the KL term was responsible for the poor reconstructions of the VAE.

III 5.2.2 Sparse encoder

The second variant replaces the dense convolutional encoder with a sparse one, following the sparse structure VAE of XCube (?). Ren et al. argue that voxel grids are flexible, can represent both bulky and thin structures and allow fast querying and processing. Sparse voxel representations go further by not storing any information for empty regions which makes them considerably more efficient (?). This is particularly relevant for airway trees since only a very small fraction of the 128^3 or 256^3 grid is occupied, so a dense encoder spends most of its computation and memory on empty space.

Encoder design.

As in XCube (?) the encoder is built from neural network operators defined over sparse voxel grids. It processes the input occupancy grid with a sparse convolutional neural network that alternates between sparse convolutions and max pooling operations reducing the resolution step by step until it reaches the resolution of the latent representation. XCube implements these operators on top of the VDB data structure (?), whose compact storage and fast look-up make sparse convolution and pooling efficient on the GPU. We implemented the sparse encoder with spconv library using the same sparse convolution layers and max pooling layers as with the dense models. The resulting sparse features are then converted to a dense 7^3 grid and mapped to the latent code in the same way as in the dense model. We kept the decoder unchanged, it is the same dense decoder as described above. The sparse-encoder model was trained with the autoencoder objective of Equation (??).

III 5.2.3 Results

Neither architecture produced satisfactory reconstructions. As with the VAE reconstructions of validation and test airway trees show missing branches, broken connectivity, blob-like structures (Figure ??).

Both models also show the same overfitting behaviour as the VAE. The training loss decreases steadily, while the validation loss does not follow it (Figure ??). Data augmentation and dropout were also applied to these models without improvement.

The deterministic autoencoder, the sparse-encoder model and the VAE all show the same pattern. This suggests that the problem is not caused by the variational formulation or by the type of encoder. More likely causes are the limited amount of training data relative to model capacity and the difficulty of reconstructing thin branching structures with a dense voxel decoder.

II 5.3 Implicit Neural Representation of Airway Trees

The voxel-based models of Section “??” and Section “??” did not produce satisfactory reconstructions. We therefore trained an implicit model: the DeepSDF auto-decoder of ?, optimized with the signed distance loss proposed for airway trees by ? instead of the original DeepSDF loss. This model gave the best reconstructions of all the architectures we tested, and its learned latent codes are the representation used for the clustering in Section “??”.

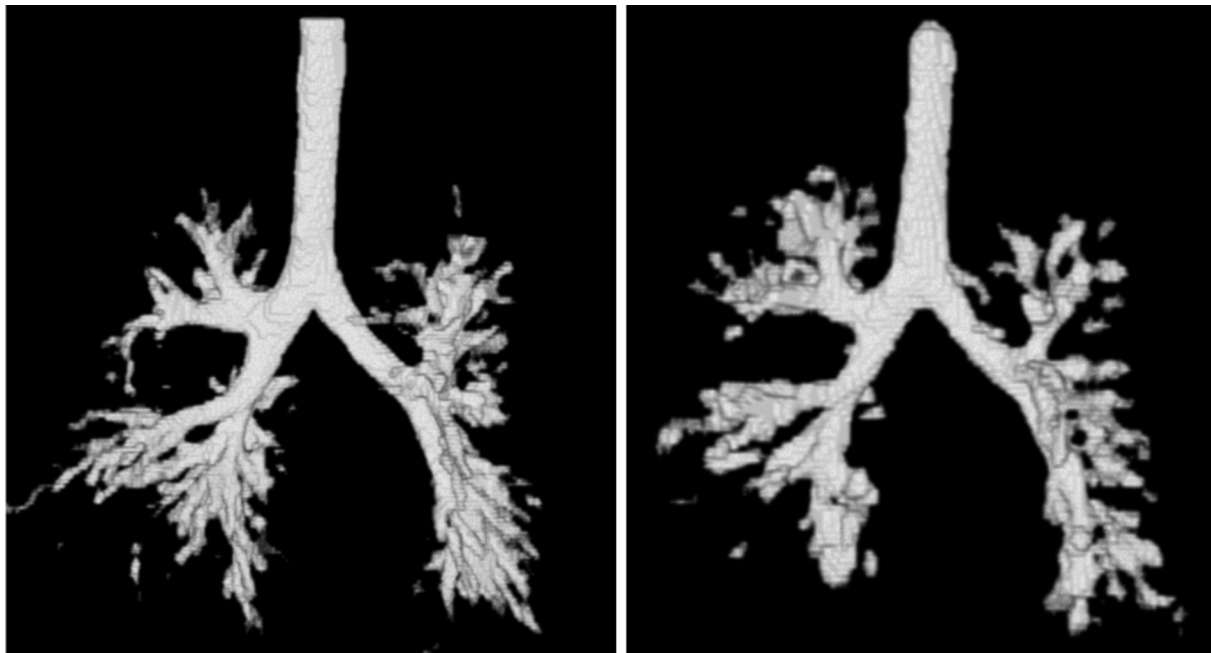


Figure 14: Reconstructions from (left) the deterministic autoencoder and (right) the sparse-encoder model samples.

5.3.1 Motivation

Airway annotations are made slice by slice in CT image stacks, so the reconstructed shape is inevitably limited by the resolution of the discrete voxels, and the image contrast degrades as the bronchi become thinner. As a result, voxel-based airway trees tend to be coarse and affected by spike-like noise, and CNN-based methods, which also work in the discrete voxel space, suffer from the same problems as well as from breakages in the tree (?).

Implicit neural representations (INRs) address this by using coordinate-based neural networks to parameterize a shape in continuous physical space instead of on a voxel grid. They are therefore not bounded by the CT grid resolution, and they yield smoother shapes with fewer spike-like artifacts (??). DeepSDF (?) learns a single network that represents a whole class of shapes, each shape being identified by a low-dimensional latent code. These codes form a compact, continuous embedding of the training shapes, which is what we need for clustering.

However, ? observed that the loss used by DeepSDF, an L1 penalty on the predicted SDF values, is not sufficient to model fine airway structures with high fidelity. They proposed additional geometric constraints, derived from implicit geometric regularization (?), as part of their Deep Geometric Correspondence Implicit (DGCI) model. We kept the DeepSDF architecture and adopted only this loss (Section “??”).

5.3.2 Model

Neural signed distance field.

Each airway tree is represented by its signed distance function (SDF), which gives, for any point in space, the distance to the closest point of the airway surface, with a negative sign inside the lumen and a positive sign outside (?). A single neural network f_θ represents the SDFs of all trees. It takes a

coordinate $\mathbf{p} \in \mathbb{R}^3$ together with a latent code $\mathbf{z}_i \in \mathbb{R}^K$ specific to the i -th airway tree, and predicts the signed distance of $\mathbf{p}$ to the surface of that tree:

$$\mathcal{F}(\mathbf{p}; \mathbf{z}_i) = f_\theta(\mathbf{z}_i, \mathbf{p}) \approx \text{SDF}_i(\mathbf{p}), \quad f_\theta : \mathbb{R}^{K+3} \rightarrow \mathbb{R}. \quad (29)$$

The airway surface of tree i is the zero level set of $\mathcal{F}(\cdot; \mathbf{z}_i)$, and can be extracted as a mesh with Marching Cubes (?).

Auto-decoder.

DeepSDF has no encoder (Figure ??). Instead, each training tree is assigned a latent code $\mathbf{z}_i$ that is a free parameter, randomly initialized and optimized jointly with the network weights θ by backpropagation (?). After training, the decoder weights are fixed, and the code of a new tree is obtained by optimizing $\mathbf{z}$ alone so that $\mathcal{F}(\cdot; \mathbf{z})$ fits the samples of that tree (Section “?”).

Network.

Following ?, f_θ is a multilayer perceptron with eight hidden fully connected layers of 512 units, each with weight normalization (?) and no dropout. The input is the concatenation $[\mathbf{z}_i, \mathbf{p}]$, and the same vector is concatenated again to the output of the fourth layer (skip connection). We use latent codes of dimension $K = 256$.

Changes required by the geometric loss.

The loss of Section “?” constrains the spatial gradient $\nabla_{\mathbf{p}} \mathcal{F}$, so training backpropagates through this gradient. We therefore made three changes to the original DeepSDF network, following implicit geometric regularization (?):

- the ReLU activations are replaced by Softplus with $\beta = 100$, a smooth approximation of ReLU whose gradient is continuous, so that the gradient terms of the loss can themselves be differentiated;
- the final $\tanh$ is removed, because it bounds $|\mathcal{F}| < 1$ and flattens the output far from the surface, which conflicts with the unit-gradient (Eikonal) constraint;
- the weights are initialized with the geometric initialization of ?, so that before training the network approximates the SDF of a sphere of radius 0.5. The weights reading the latent code start at zero, so every tree starts from the same sphere.

III 5.3.3 Loss function

Sample sets.

For each tree i of a mini-batch $\mathcal{B}$, three sets of points are drawn in every iteration (Section “?”): surface points S_i , with their ground-truth outward normals $\mathbf{n}'$; free-space points V_i near and away from the surface, with their ground-truth signed distances s' ; and points U_i drawn uniformly in the normalized volume $[-1, 1]^3$, which have no ground truth. On the surface, $s' = 0$. We write $\Omega_i = S_i \cup V_i \cup U_i$ for

[DeepSDF auto-decoder figure]

Figure 15: The DeepSDF auto-decoder, adapted from ?. The latent code $\mathbf{z}_i \in \mathbb{R}^{256}$ of tree i is concatenated with a query point $\mathbf{p}$ and passed through eight fully connected layers of width 512, with the input concatenated again after the fourth layer, to predict the signed distance $\mathcal{F}(\mathbf{p}; \mathbf{z}_i)$. The codes have no encoder: they are optimized together with the network weights.

all points of tree i , and, for a family of sets A_i , $|A| = \sum_{i \in \mathcal{B}} |A_i|$ for its total number of points in the mini-batch.

SDF loss.

We use the SDF loss of ?, written here without their shared code, so that $\mathcal{F}$ depends only on $\mathbf{z}_i$. It has two parts, $\mathcal{L}_{\text{sdf}} = \mathcal{L}_{\text{sdf-val}} + \mathcal{L}_{\text{sdf-geo}}$, which supervise the values and the geometry of the predicted SDF respectively:

$$\mathcal{L}_{\text{sdf-val}} = \frac{\omega_s}{|S \cup V|} \sum_{i \in \mathcal{B}} \sum_{\mathbf{p} \in S_i \cup V_i} |\mathcal{F}(\mathbf{p}; \mathbf{z}_i) - s'| + \frac{\omega_\varphi}{|U|} \sum_{i \in \mathcal{B}} \sum_{\mathbf{p} \in U_i} \varphi(\mathcal{F}(\mathbf{p}; \mathbf{z}_i)), \quad (30)$$

$$\begin{aligned} \mathcal{L}_{\text{sdf-geo}} = & \frac{\omega_n}{|S|} \sum_{i \in \mathcal{B}} \sum_{\mathbf{p} \in S_i} (1 - S_{\cos}(\nabla \mathcal{F}(\mathbf{p}; \mathbf{z}_i), \mathbf{n}')) \\ & + \frac{\omega_{\text{Eik}}}{|\Omega|} \sum_{i \in \mathcal{B}} \sum_{\mathbf{p} \in \Omega_i} \left| \|\nabla \mathcal{F}(\mathbf{p}; \mathbf{z}_i)\|_2 - 1 \right|, \end{aligned} \quad (31)$$

where $S_{\cos}$ is the cosine similarity and the gradient is taken with respect to $\mathbf{p}$. Unlike ?, who sum over the points, we divide each term by the number of points of its set in the mini-batch, so every term is a mean and the weights do not depend on the number of samples.

The first term of Equation (??) regresses the SDF values directly. Unlike DeepSDF, which clamps both prediction and target to $[-0.1, 0.1]$ (?), it is applied without clamping. The second term, with $\varphi(s) = \exp(-\delta|s|)$ and $\delta = 100$, penalizes points away from the surface whose predicted SDF is close to zero, and therefore suppresses spurious surfaces in empty space (?). ? write this term over all off-surface points; we apply it, as in their released code, only to the uniform points U_i , because many free-space points V_i lie very close to the surface, where a small SDF is correct and should not be penalized. Equation (??) aligns the SDF gradient with the surface normals and constrains its norm to one everywhere, as required for a true distance function (Eikonal equation) (??).

Latent code regularization.

? regularize their instance and shared codes and add a correspondence loss for their encoder–decoder. Since our model has neither a shared code nor an encoder–decoder, we keep instead the code

regularization of the DeepSDF implementation, which corresponds to a zero-mean Gaussian prior on the codes (?):

$$\mathcal{L}_{\text{reg}} = \lambda \min\left(1, \frac{t}{100}\right) \frac{1}{B} \sum_{i \in \mathcal{B}} \|\mathbf{z}_i\|_2, \quad (32)$$

where $\mathcal{B}$ is the mini-batch of B trees, t the epoch, and $\lambda = 0.3$. The factor $\min(1, t/100)$ increases the regularization linearly over the first 100 epochs. In addition, the norm of every code is capped at 1.

Total objective.

The decoder weights θ and all training codes $\{\mathbf{z}_i\}$ are optimized jointly by minimizing

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{sdf-val}} + \mathcal{L}_{\text{sdf-geo}} + \mathcal{L}_{\text{reg}}. \quad (33)$$

Following ?, we set $\omega_s = 3 \times 10^3$, $\omega_\varphi = 5 \times 10^2$, $\omega_n = 10^2$ and $\omega_{\text{Eik}} = 5 \times 10^1$.

III 5.3.4 Data preparation and training

Data.

Our dataset contains [TODO: N] airway trees from the AIB23 [TODO: (?)] and ATM22 [TODO: (?)] datasets, split into [TODO: N_{train}] training, [TODO: N_{val}] validation and [TODO: N_{test}] test cases. Each airway surface was extracted from its binary segmentation with Marching Cubes (?), and the SDF samples were prepared with the preprocessing of DeepSDF (?). Each mesh is centered on its bounding box and scaled to fit in the unit sphere with a margin of 3%; the offset and scale are stored, so that predictions can be mapped back to CT coordinates. Then, for each tree:

- 200,000 surface points with outward normals are obtained by rendering the mesh from virtual cameras placed around it and back-projecting the depth maps;
- 500,000 SDF samples are generated: 94% near the surface, by perturbing surface points with Gaussian noise of variance 5×10^{-4} and 5×10^{-5} , and 6% uniformly in $[-1, 1]^3$. Their signed distance is the distance to the closest surface point, with the sign given by the orientation of the closest normals.

Training.

In every iteration, we draw for each tree 4,096 surface points (S_i), 4,096 free-space points (V_i), half of them inside and half outside the airway, and 2,048 uniform points (U_i). Balancing inside and outside samples is important for airways, since the lumen occupies only a small fraction of the normalized volume and most samples are naturally outside. The mini-batch contains $B = 64$ trees. The decoder weights and the latent codes are optimized with Adam (?), with initial learning rates of 10^{-3} and 2×10^{-3} respectively, both halved every 4,000 epochs. The codes are initialized from $\mathcal{N}(0, 1/K)$. The model was trained for [TODO: E] epochs on a single [TODO: NVIDIA A100 / L40S] GPU. The training losses are shown in Figure ??.

After training, each training tree is represented by its latent code $\mathbf{z}_i \in \mathbb{R}^{256}$. These codes are the features used for clustering in Section “??”.

Latent codes of unseen trees.

For validation and test trees, the decoder weights are frozen and only a new code $\mathbf{z}$ is optimized (?). The code is initialized from $\mathcal{N}(0, 0.01^2)$ and optimized for 800 iterations with Adam, with a learning rate of 5×10^{-3} divided by 10 after 400 iterations. The objective is the same SDF loss, $\mathcal{L}_{\text{sdf-val}} + \mathcal{L}_{\text{sdf-geo}}$, on freshly drawn samples of the tree, plus a prior term $10^{-4} \omega_s \overline{\mathbf{z}^2}$, where $\overline{\mathbf{z}^2}$ is the mean of the squared code entries.

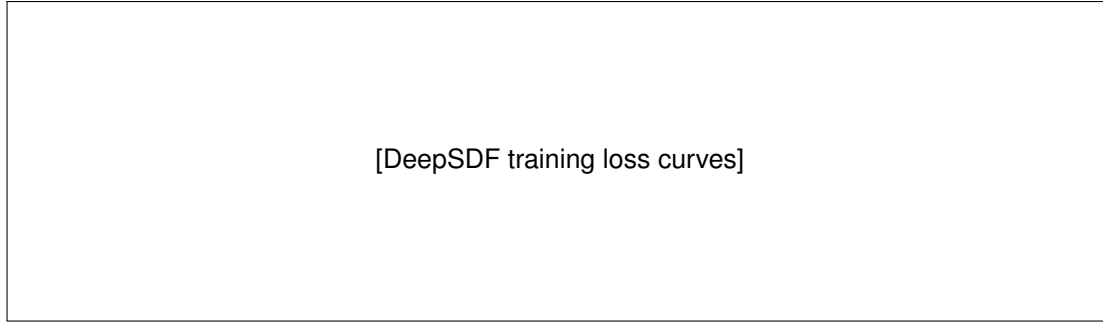


Figure 16: Training losses of the DeepSDF model: total loss and the weighted terms of Equations (??)–(??) (SDF value, φ , normal, Eikonal and code regularization).

III 5.3.5 Evaluation metrics

We follow the evaluation protocol of ?, which measures shape quality both in continuous space, on the mesh extracted from the predicted SDF, and in discrete voxel space. Meshes are extracted with Marching Cubes on a [TODO: 256]³ grid in the normalized volume.

Continuous space.

Smoothness and connectivity of the reconstructed mesh are measured by counting non-manifold (NM) elements. NM-V counts the vertices connected to three or more non-coplanar faces, NM-E counts the edges shared by three or more faces, and NM-F counts the faces arranged in a way that prevents a closed, continuous body from being formed. Fewer non-manifold elements indicate better connectivity and smoothness (?).

Discrete space.

To compute voxel-based metrics, the predicted SDF is evaluated directly on the voxel grid of the original CT image: the center of each voxel is mapped to the normalized volume with the stored offset and scale, and the voxel is labeled as airway if the predicted SDF is negative [TODO: (with $2 \times$ supersampling near the surface and a 50% occupancy threshold)]. On this volume we compute the tree length detected rate (TD) and the branch detected rate (BD), defined in Eq. (??), and the Dice coefficient, which measure the topological completeness and correctness of the reconstruction (?).

III 5.3.6 Results

Table ?? reports the metrics on the [TODO: validation and test] sets, and Figure ?? shows examples of reconstructed airway trees. This model produced the best reconstructions of all the models we trained. [TODO: Describe the main observations, e.g. preservation of fine distal branches, connectivity of the tree, smoothness of the surface.]

Table 8: Reconstruction results of the DeepSDF model trained with the SDF loss of ?, reported as mean $\pm$ standard deviation.

Set	Continuous space			Discrete space		
	NM-V $\downarrow$	NM-E $\downarrow$	NM-F $\downarrow$	TD (%) $\uparrow$	BD (%) $\uparrow$	Dice (%) $\uparrow$
Validation	[TODO: ?]	[TODO: ?]	[TODO: ?]	[TODO: ?]	[TODO: ?]	[TODO: ?]
Test	[TODO: ?]	[TODO: ?]	[TODO: ?]	[TODO: ?]	[TODO: ?]	[TODO: ?]

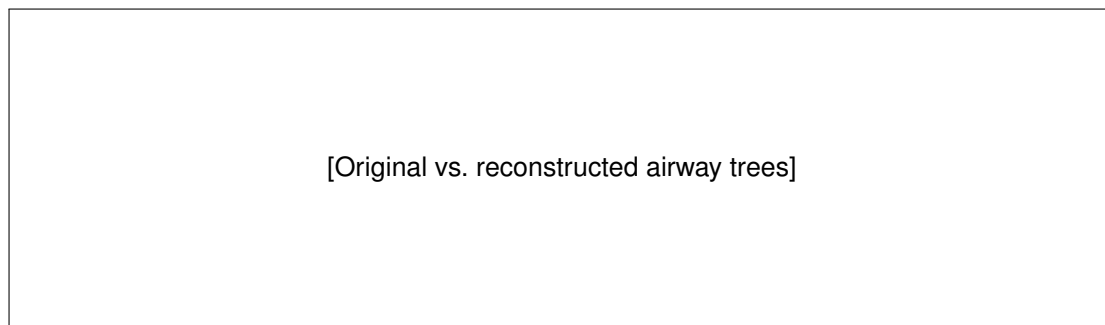


Figure 17: Ground-truth airway trees and their reconstructions by the DeepSDF model.

I 6 Anatomical Labeling of Airway Trees

To assign anatomical labels to our airway trees, we used the Implicit Point-Graph Network (IPGN) of ?. We did not train this model ourselves: we used the weights released by the authors, trained on their Pulmonary Tree Labeling (PTL) dataset, and ran the model in inference mode only. This section summarizes the label definition, the model and its reported performance, and then describes how we applied it to our data.

II 6.1 Task formulation

The input to the model is a binary volume containing a pulmonary tree (airway, artery or vein). The goal is a 19-class semantic segmentation of every foreground voxel: 18 classes correspond to segmental-level branches, i.e. the branches that supply the 18 pulmonary segments, and one class corresponds to the extra-pulmonary structures (?). Labels 1 to 10 are branches in the left lung, labels 11 to 18 are branches in the right lung, and label 19 is the extra-pulmonary part (?). The correspondence between label indices and segmental bronchi is given in Table ??.

Since branching points and distal end points contain very few voxels, a high voxel-level score does not guarantee correct labels at these key locations; IPGN therefore also represents each tree as a skeleton graph whose nodes approximate the key points (?).

Table 9: Anatomical classes predicted by IPGN (?). [TODO: Fill names from the PTL label definition.]

Label	Left lung	Label	Right lung
1	[TODO:]	11	[TODO:]
2	[TODO:]	12	[TODO:]
3	[TODO:]	13	[TODO:]
4	[TODO:]	14	[TODO:]
5	[TODO:]	15	[TODO:]
6	[TODO:]	16	[TODO:]
7	[TODO:]	17	[TODO:]
8	[TODO:]	18	[TODO:]
9	[TODO:]		
10	[TODO:]		
19	Extra-pulmonary structures		

6.2 Model

Input representations.

Instead of processing the dense volume with a CNN, IPGN works on two sparse representations of the tree (?). The first is a point cloud of $M = 6,000$ randomly sampled foreground voxels. The second is a skeleton graph built with the VesselVio software (?), which applies the thinning algorithm of ? to extract the centerlines: centerline segments become edges, and centerline intersections and distal end points become nodes. The only input features are the (x, y, z) coordinates of the points and nodes.

Architecture.

The model has three stages (Figure ??) (?). First, a point encoder and an 11-layer graph attention network (GAT) (?) compute an initial feature for each point and each graph node. The point encoder is either PointNet++ (?) or Point Transformer (?); the released model we used has a [TODO: PointNet++ / Point Transformer] encoder. Second, a stack of Point-Graph Fusion layers exchanges information between the two branches: each graph node aggregates the features of the points within a small ball around it, and each point aggregates the features of its nearest graph nodes, weighted by inverse distance. In this way, the points receive the structural context of the skeleton and the nodes receive local shape information. Third, the Implicit Point Module labels every foreground voxel, not only the 6,000 sampled points: for any query location, it interpolates the features of the nearest sampled points from all fusion stages and maps them to 19 class scores with a small MLP. The network without this module is called the Point-Graph Network (PGN).

6.3 Training data and reported performance

The released weights were trained on the PTL dataset (?), which contains 800 subjects from multiple medical centers, each with an airway, artery and vein volume annotated with the 19 classes by a junior radiologist and verified by a senior radiologist. The CT volumes have 512×512 slices, between 181 and 798 slices per scan, and a slice spacing between 0.5 and 1.5 mm. The point and graph encoders were first pre-trained separately and frozen; the fusion layers and the Implicit Point Module were then trained jointly with a cross-entropy loss, using random rotation, shifting and scaling as data augmentation (?).

[IPGN overview figure]

Figure 18: Overview of the Implicit Point-Graph Network (IPGN), adapted from ?. The binary volume is converted into a skeleton graph and a point cloud, Point-Graph Fusion layers exchange features between the two branches, and the Implicit Point Module produces the dense labeled volume.

Performance is measured with micro-averaged Dice scores at two levels (?): at the *voxel level*, on the labeled volume, reflecting overall performance; and at the *graph level*, on the labels of the skeleton nodes and edges, reflecting performance at the key locations of the tree. On the PTL test set, IPGN with the [TODO: PointNet++ / Point Transformer] encoder reaches a voxel-level Dice of [TODO: 87.5 / 88.5] % on airways, and [TODO: 95.1 / 95.0] % and [TODO: 92.3 / 92.7] % on graph nodes and edges, respectively, outperforming CNN-, point- and graph-based baselines (?). On a version of PTL with artificially disconnected trees, its voxel-level Dice dropped by only 1.2 points on average (?), which is relevant if the input segmentations are imperfect.

II 6.4 Application to our airway trees

Pre-processing.

We applied the released IPGN model with the [TODO: PointNet++ / Point Transformer] encoder to our [TODO: number] airway trees, obtained by [TODO: manual / automatic segmentation with ...]. Each tree was [TODO: binarized and resampled to ...]. Because labels 1–10 and 11–18 encode the left and right lung, the volumes were [TODO: reoriented to the PTL axis convention ...] so that left and right are not swapped.

Inference.

Following ?, each binary airway volume was skeletonized with VesselVio, and 6,000 foreground voxels were randomly sampled as the point cloud. A single forward pass of PGN on the graph and point cloud gives the multi-stage point features, after which all remaining foreground voxels are passed through the Implicit Point Module as query points, giving a label for every voxel of the tree. [TODO: Optional: runtime on our hardware, e.g. $\approx \dots$ s per tree.]

Quality control.

Our data has no ground-truth segmental labels, so the reported Dice scores cannot be verified directly. The predictions were instead checked by [TODO: visual inspection of all / a random sample of ... trees], verifying that [TODO: each label forms a single connected subtree, all 18 segmental classes are present, and left- and right-lung labels lie on the correct side]. Trees with [TODO: evident

errors were excluded / manually corrected / kept, [TODO: n] in total. Examples of labeled trees are shown in Figure ??.

Limitations.

The performance figures above were obtained on PTL, and our data differ from it in [TODO: scanner / resolution / population / segmentation method]. The labels should therefore be regarded as [TODO: ...]. The resulting labels are used in Section “??” to compute the morphological descriptors of each anatomical class.

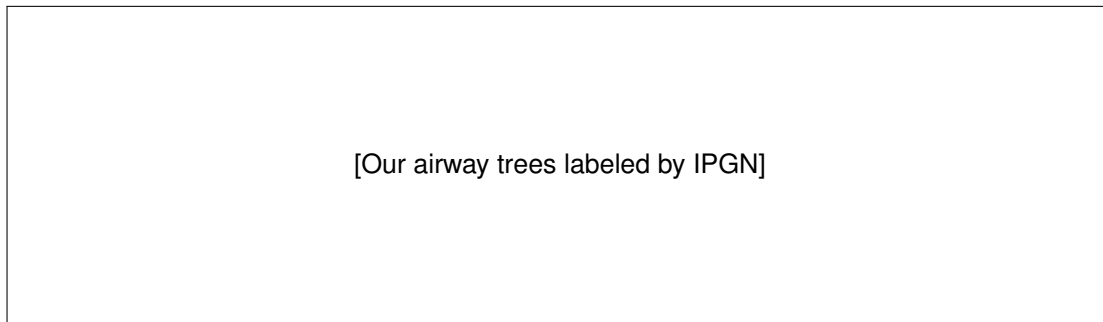


Figure 19: Anatomical labels predicted by IPGN on examples of our airway trees. Colours denote the 19 classes listed in Table ??.

I 7 Morphological Analysis of Airway Trees

Once every airway tree is anatomically labeled (Section “??”), we describe its shape with the six morphological descriptors of AirwaySignature, introduced with the AirMorph framework by ? : Stenosis (S), Ectasia (E), Tortuosity (T), Length (L), Divergence (D) and Complexity (C). The descriptors capture local variations in airway shape that cannot be seen from the binary segmentation alone, and they are defined per anatomical class, so that each value refers to a specific bronchus (a segmental or a lobar branch) (?). For each tree, the result is a compact, anatomically aligned signature: one value of each descriptor for every anatomical class present in the tree.

II 7.1 Overview

? divide the six descriptors into two groups (Figure ??):

- **Branch-level descriptors** (S , E , T) are first computed on each individual branch of the airway skeleton and then averaged over all branches that belong to the same anatomical class. Stenosis measures the strongest narrowing of a branch, Ectasia its strongest dilation, and Tortuosity how much its 3D trajectory bends.
- **Subtree-level descriptors** (L , D , C) are computed on the whole subtree of an anatomical class, i.e. on all connected branches that share the same anatomical context. Length measures the geodesic length of the subtree, Divergence the angular spread of its terminal branches, and Complexity the irregularity and density of its branching through a fractal dimension.

The anatomical class K over which the descriptors are grouped can be a segmental or a lobar bronchus

(?). In our case the labels come from IPGN, which provides 18 segmental classes; **[TODO: lobar classes were obtained by merging the segmental labels of each lobe / we computed the descriptors at segmental level only]**.

Table ?? summarizes the six descriptors.

Table 10: The six AirwaySignature descriptors of ?.

Descriptor	Computed on	Range	Higher value means
Stenosis (S)	Each branch, then averaged	$[0\%, 100\%]$	stronger narrowing
Ectasia (E)	Each branch, then averaged	$[100\%, +\infty)$	stronger dilation
Tortuosity (T)	Each branch, then averaged	$[0, 1]$	stronger bending
Length (L)	Class subtree	$[0, +\infty)$ mm	longer subtree
Divergence (D)	Class subtree	$[0, 1]$	wider angular spread
Complexity (C)	Class subtree	$[0, +\infty)$	more complex branching

[Definition of the six morphological descriptors]

Figure 20: The six morphological descriptors of AirwaySignature, adapted from ?.

II 7.2 Common pre-processing

Stenosis and Ectasia are both based on the local airway radius along the centerline, which is obtained as follows (?):

1. **Radius field.** A 3D Euclidean distance transform (EDT) is applied to the binary airway mask V_{airway} , taking the voxel spacing s into account. At every foreground voxel it gives the distance to the nearest background voxel, i.e. an approximation of the local airway radius in mm.
2. **Centerline radii.** The distance field is masked with the airway skeleton S , keeping only the values on centerline voxels. This gives the radius field along the skeleton:

$$R = \text{EDT}(V_{\text{airway}}; s) \cdot S. \quad (34)$$

3. **Skeleton parsing.** The skeleton is split into individual branches using a 26-connected neighborhood and removing the junction voxels, so that each branch is spatially isolated. Each parsed branch b receives the anatomical label of the region it lies in.

For every parsed branch b , the minimum, mean and maximum radius along its centerline are then computed:

$$r_{\min}^{(b)} = \min_{x \in b} R(x), \quad r_{\text{mean}}^{(b)} = \frac{1}{|b|} \sum_{x \in b} R(x), \quad r_{\max}^{(b)} = \max_{x \in b} R(x). \quad (35)$$

7.3 Branch-level descriptors

Stenosis (S).

Stenosis quantifies airway narrowing. We define it as the reduction of the radius at the point of strongest constriction, R_{narrow} , relative to a reference radius R_{ref} , the mean radius of a healthy proximal airway segment:

$$\text{Stenosis} = \left(1 - \frac{R_{\text{narrow}}}{R_{\text{ref}}}\right) \times 100\%. \quad (36)$$

In their implementation, both quantities are taken from the radius statistics of each parsed branch: R_{narrow} is the branch's minimum radius and R_{ref} its mean radius, which gives the local stenosis ratio of branch b :

$$S_b = 1 - \frac{r_{\min}^{(b)}}{r_{\text{mean}}^{(b)}}. \quad (37)$$

The stenosis of an anatomical class K is the average over all branches carrying that label:

$$S_K = \frac{1}{|B_K|} \sum_{b \in B_K} S_b, \quad (38)$$

where B_K is the set of parsed branches with label K . If a class is absent from the tree, its score is set to -1 (?). Stenosis ranges from 0% to 100%, and higher values mean a more severe narrowing.

Ectasia (E).

Ectasia complements Stenosis: instead of narrowing, it captures over-expansion of the airway lumen (?). It is defined as the ratio between the maximum radius and the reference radius, which in the implementation is again the mean radius of each parsed branch:

$$\text{Ectasia} = \frac{R_{\text{max}}}{R_{\text{ref}}} \times 100\%, \quad E_b = \frac{r_{\text{max}}^{(b)}}{r_{\text{mean}}^{(b)}}. \quad (39)$$

As for Stenosis, the class value E_K is the average of E_b over all branches with label K , and absent classes receive -1 . Ectasia ranges from 100% upwards, and higher values mean a stronger dilation (?).

Tortuosity (T).

Tortuosity describes the local curvature of a branch from its volumetric geometry (?). For a branch region Ω_l (the set of its voxels):

1. the voxel coordinates are converted to physical space using the voxel spacing;
2. principal component analysis (PCA) is applied to these coordinates, and the two endpoints $\mathbf{S}$ and $\mathbf{E}$ of the branch are taken along the first principal axis, i.e. the main direction of the branch;
3. the voxel $\mathbf{P}$ that lies furthest from the segment $\overline{SE}$ is found:

$$\mathbf{P} = \arg \max_{\mathbf{x}_i \in \Omega_l} \text{dist}(\mathbf{x}_i, \overline{SE}); \quad (40)$$

4. the tortuosity is the angle formed at $\mathbf{P}$ by the vectors pointing to the two endpoints:

$$T = \alpha = \angle(\overrightarrow{PS}, \overrightarrow{PE}) = \arccos \frac{\overrightarrow{PS} \cdot \overrightarrow{PE}}{\|\overrightarrow{PS}\| \|\overrightarrow{PE}\|}. \quad (41)$$

This angle reflects how much the branch bends or deflects. It lies in $[0, \pi]$ and is then normalized to $[0, 1]$, with values close to 1 indicating a high degree of tortuosity (?). **[TODO: State the normalization you used, e.g. $T = 1 - \alpha/\pi$.]** As for Stenosis and Ectasia, the class value T_K is the average over all branches with label K .

7.4 Subtree-level descriptors

The remaining three descriptors work on the centerline tree, in which each node carries an anatomical label, a generation number (its depth in the tree) and a length. For each class K , they first identify all terminal (leaf) nodes labeled K , denoted T_K , and their lowest common ancestor (LCA), found by a generation-aware traversal of the tree (?). The LCA is the node where the subtree of class K begins.

Length (L).

Length measures the geometric elongation of a segment or lobe as a geodesic length, accumulated branch by branch along the skeleton (?). If the LCA itself belongs to class K , the length is the average, over all leaves of class K , of the path length from the LCA to that leaf:

$$L_K = \frac{1}{|T_K|} \sum_{v \in T_K} \sum_{n \in \text{Path}(\text{LCA}, v)} \ell(n), \quad (42)$$

where $\text{Path}(\text{LCA}, v)$ is the set of nodes on the path from the LCA to leaf v and $\ell(n)$ is the length of node n . If the LCA does not belong to class K (for example, when it is the trachea), each path is re-rooted at the nearest downstream node labeled K , so that only the part of the tree belonging to class K is measured (?). Length is expressed in mm and ranges from 0 upwards.

Divergence (D).

Divergence measures the spatial dispersion of the branches of a class, i.e. how widely they spread from their common origin, using the smallest cone that encloses them (?). The LCA of the class- K leaves is taken as the apex of the cone, and $\mathbf{v}_i$ is the normalized direction vector from the apex to leaf i . The axis of the cone is the unit vector $\mathbf{u}^*$ that maximizes the smallest cosine similarity with all leaf directions:

$$\mathbf{u}^* = \arg \max_{\|\mathbf{u}\|=1} \min_i \langle \mathbf{u}, \mathbf{v}_i \rangle. \quad (43)$$

The divergence angle is twice the largest angular deviation of a leaf direction from this axis, i.e. the full opening angle of the enclosing cone:

$$\theta = 2 \cdot \arccos \left(\min_i \langle \mathbf{u}^*, \mathbf{v}_i \rangle \right). \quad (44)$$

The angle lies in $[0, \pi]$ and is normalized to $[0, 1]$. A smaller angle means that the branches of the class are less separated and less angularly dispersed, which can be affected by pulmonary disease (?).

Complexity (C).

Complexity measures the geometric complexity of the branches of a class with a box-counting fractal dimension, which captures the spatial irregularity and branching density of the skeleton (?). For each class K :

1. the skeleton voxels labeled K are extracted as a binary mask, $\mathcal{S}_K = (S > 0) \wedge (Y = K)$;
2. this mask is cropped to its bounding box and zero-padded to a fixed 3D size;
3. the volume is covered with cubic boxes of sizes $s \in \{2^1, 2^2, \dots, 2^k\}$, and for each size the number $N(s)$ of boxes containing at least one skeleton voxel is counted.

The complexity is the box-counting dimension

$$C = \lim_{s \rightarrow 0} \frac{\log N(s)}{\log(1/s)}, \quad (45)$$

estimated in practice as the slope of a linear regression of $\log N(s)$ against $\log(1/s)$ (?). A straight line has a dimension close to 1, while a densely branching structure that fills more of the space has a higher value. Complexity ranges from 0 upwards, and larger values denote a more complex branching pattern.

II 7.5 Application to our airway trees

We computed the six descriptors for all [TODO: number] labeled airway trees, at [TODO: segmental / lobar] level, following the procedures above. [TODO: Describe any implementation choices, e.g. the skeletonization method, the normalization of T and D , the node lengths $\ell(n)$, and the maximum box size 2^k .] Examples of the resulting signatures are shown in Figure ???. These descriptors are used in [TODO: Section “??”] to [TODO: describe their use, e.g. to characterize the clusters].

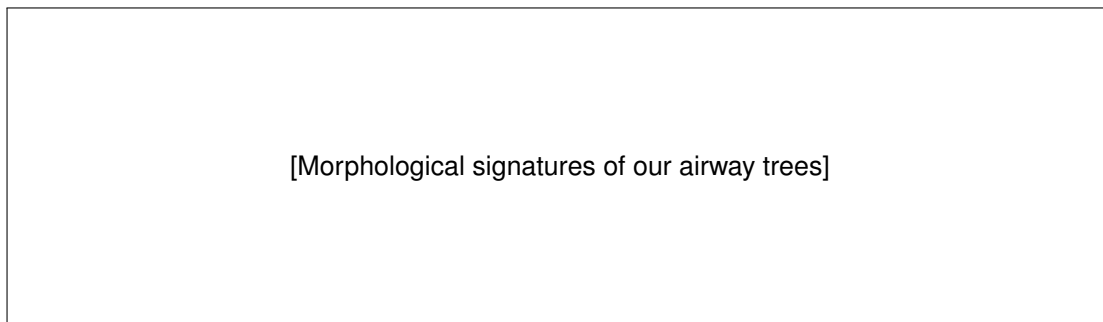


Figure 21: Morphological descriptors computed on examples of our labeled airway trees.

I 8 Clustering

The implicit model of represents every airway tree by a latent code. If groups of trees with a similar shape exist, they should appear as groups of nearby codes. Following the phenotype-discovery approach of ?, who cluster the latent codes of a 3D autoencoder to find imaging phenotypes without any labels, we cluster the latent codes of the airway trees with two families of methods: agglomerative hierarchical clustering and K-means, the latter also in a kernel version. The groups found are then visualized with three projections of the latent space and characterized with the morphological descriptors of Section “??” (Figure ??).

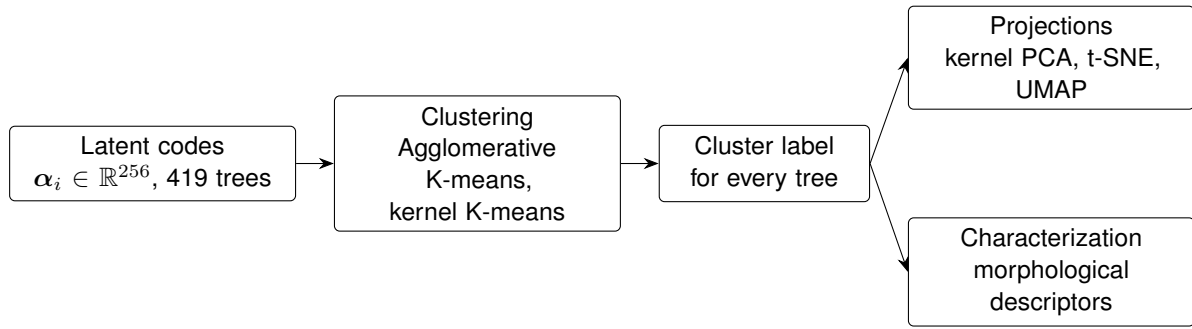


Figure 22: Clustering pipeline. The latent codes are clustered in their full space; the projections are only used to look at the result, and the morphological descriptors to describe what distinguishes the clusters.

8.1 Agglomerative hierarchical clustering

Agglomerative clustering builds a hierarchy of nested groups from the bottom up (?). At the start, every tree is a cluster of its own. At each step, the two closest clusters are merged, and the procedure repeats until a single cluster contains all the trees. The sequence of merges forms a binary tree, the *dendrogram*, in which the height of each junction shows how far apart the two merged clusters were. Cutting the dendrogram with a horizontal line at some height gives a partition into K clusters, so one run of the algorithm yields the clustering for every K and shows how small groups nest inside larger ones. This is what makes hierarchical methods attractive for discovering intrinsic groupings whose number is not known in advance (?).

Linkage

What “closest” means is set by the linkage. ? merge the two clusters whose centroids are closest. We use Ward’s linkage (?), which merges the pair of clusters whose union increases the total within-cluster sum of squares the least. For two clusters A and B with centroids μ_A and μ_B , this increase is

$$\Delta(A, B) = \frac{|A||B|}{|A| + |B|} \|\mu_A - \mu_B\|_2^2. \quad (46)$$

Like the centroid linkage, Ward’s criterion compares cluster centres, but the factor in front of the distance penalizes merging two large clusters, so it favours compact clusters of similar size. Since it greedily minimizes the same within-cluster sum of squares as K-means (Eq. (??)), the two methods can be compared directly.

Choosing the number of clusters

To choose where to cut the dendrogram, we follow the elbow criterion of ?. For every number of clusters $k = 1, \dots, 19$, we cut the hierarchy into k clusters and compute the average intra-cluster distance, the mean distance between two trees of the same cluster, averaged over the clusters:

$$D_{\text{avg}}(k) = \frac{1}{k} \sum_{l=1}^k \frac{1}{N_l^2} \sum_{u=1}^{N_l} \sum_{v=1}^{N_l} \|\mathbf{x}_{ul} - \mathbf{x}_{vl}\|_2, \quad (47)$$

where N_l is the number of trees in cluster l and $\mathbf{x}_{ul}$ is the code of its u -th tree. Splitting a cluster always makes its parts more compact, so D_{avg} tends to decrease with k . The useful information is where it

drops the most: the number of clusters is chosen as the one with the largest relative reduction (?),

$$k^* = \arg \max_k \frac{D_{\text{avg}}(k-1) - D_{\text{avg}}(k)}{D_{\text{avg}}(k-1)}. \quad (48)$$

A large relative drop means that going from $k-1$ to k clusters separates groups that are genuinely different; after that point, further splits mostly cut coherent groups into pieces. The elbow criterion identified $k=4$ as the optimal number of clusters for both the registered and unregistered airway trees. Thus, in both representations, the hierarchy was cut at four clusters for the subsequent analysis.

8.2 K-means

Unlike agglomerative clustering, K-means directly partitions the trees into a fixed number K of clusters. Each cluster c is represented by its centroid μ_c , and the partition is chosen to minimize the *inertia*, the total squared distance of the trees to the centroid of their cluster:

$$J = \sum_{i=1}^N \|\mathbf{x}_i - \mu_{c(i)}\|_2^2, \quad (49)$$

where $c(i)$ is the cluster of tree i . Lloyd's algorithm (?) alternates two steps: every tree is assigned to its nearest centroid, then every centroid is moved to the mean of the trees assigned to it. Neither step can increase J , so the algorithm converges, but only to a local minimum that depends on the initial centroids. We therefore initialize the centroids with k-means++ (?), which spreads them out by picking distant trees with higher probability, and keep the best of 20 runs with different initializations (scikit-learn implementation (?)). Because each tree joins its nearest centroid, the borders between clusters are flat hyperplanes: K-means works best for compact, roughly spherical clusters.

Choosing the number of clusters

K-means requires the number of clusters K to be specified in advance. We therefore evaluated $K = 2, \dots, 10$ for each airway-tree representation. For each candidate K , K-means was fitted to the 256-dimensional latent codes using k-means++ initialization with 20 restarts, retaining the solution with the lowest inertia. Two criteria were then evaluated: the inertia, which measures the within-cluster sum of squared distances, and the silhouette coefficient (?), which measures how well each tree is separated from the other clusters. The mean silhouette was used to assess the separation of the resulting partition.

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \in [-1, 1]. \quad (50)$$

The analysis identified $K=3$ as the preferred number of clusters for both the registered and unregistered airway-tree representations. The final K-means analysis was therefore performed with three clusters for both representations.

8.3 Visualizing the latent space

The latent codes are represented in a 256-dimensional space and therefore cannot be directly visualized. We project them to two and three dimensions using three complementary dimensionality-reduction methods and colour the resulting embeddings according to cluster membership or morphological descriptors. The three methods distort the data in different ways, which is why we compare them.

Kernel PCA

Principal component analysis (PCA) finds the orthogonal directions of largest variance; kernel PCA (?) does the same in the feature space of a kernel. With the RBF kernel of Eq. (??) and the same median-rule γ as kernel K-means, it shows the space in which kernel K-means operates. The kernel matrix K is first centred, $\tilde{K} = HKH$ with $H = I - \frac{1}{N}\mathbf{1}\mathbf{1}^\top$, and decomposed as $\tilde{K}\mathbf{v}_m = \lambda_m\mathbf{v}_m$. The coordinate of tree i on the m -th component is $\sqrt{\lambda_m}v_{m,i}$, and the share of kernel variance kept by a view with d components is $\sum_{m \leq d} \lambda_m / \sum_m \lambda_m$.

t-SNE

t-distributed stochastic neighbour embedding (t-SNE) (?) preserves neighbourhoods rather than distances. For every tree i , it turns distances into probabilities of picking tree j as a neighbour,

$$p_{j|i} = \frac{\exp(-\|\mathbf{x}_i - \mathbf{x}_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|\mathbf{x}_i - \mathbf{x}_k\|^2 / 2\sigma_i^2)}, \quad (51)$$

where the width σ_i is chosen so that the distribution has a given *perplexity*, roughly the effective number of neighbours. In the projection, similarities q_{ij} follow a heavy-tailed Student- t distribution, $q_{ij} \propto (1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2)^{-1}$, and the points $\mathbf{y}_i$ are moved by gradient descent to minimize the Kullback–Leibler divergence between the symmetrized p_{ij} and q_{ij} . The heavy tail lets dissimilar trees drift apart, which gives well-separated groups, but the sizes of the groups and the distances between them are not meaningful. We ran t-SNE with PCA initialization and 1000 iterations, for perplexities from 5 to 50 in steps of 5.

UMAP

Uniform Manifold Approximation and Projection (UMAP) (?) is a nonlinear dimensionality-reduction method that seeks to preserve the neighbourhood structure of the high-dimensional data in a lower-dimensional representation. Given the latent codes $\mathbf{x}_1, \dots, \mathbf{x}_N$, UMAP first identifies the nearest neighbours of each observation using the Euclidean distance. These neighbourhood relationships are represented by a weighted graph.

For each tree i , UMAP determines a local connectivity distance ρ_i , corresponding to the distance to the closest neighbour, and a local scale parameter σ_i . The membership strength of the connection between tree i and one of its neighbours j is defined as

$$w_{ij} = \begin{cases} 1, & d(\mathbf{x}_i, \mathbf{x}_j) \leq \rho_i, \\ \exp\left(-\frac{d(\mathbf{x}_i, \mathbf{x}_j) - \rho_i}{\sigma_i}\right), & d(\mathbf{x}_i, \mathbf{x}_j) > \rho_i, \end{cases} \quad (52)$$

where $d(\mathbf{x}_i, \mathbf{x}_j)$ denotes the Euclidean distance between two latent codes. Thus, nearby observations receive a high connection strength, while more distant neighbours have smaller weights. The local scales are chosen so that the graph adapts to the density of the data around each observation.

UMAP then seeks a low-dimensional representation $\mathbf{y}_1, \dots, \mathbf{y}_N$ whose neighbourhood structure reproduces that of the original graph. In the low-dimensional space, the strength of an association between two observations is modelled by

$$\tilde{w}_{ij} = \frac{1}{1 + a\|\mathbf{y}_i - \mathbf{y}_j\|_2^{2b}}, \quad (53)$$

where a and b determine the shape of the low-dimensional decay function. The embedding is obtained by minimizing the cross-entropy between the high- and low-dimensional fuzzy graphs:

$$\mathcal{L}_{\text{UMAP}} = - \sum_{i,j} [w_{ij} \log(\tilde{w}_{ij}) + (1 - w_{ij}) \log(1 - \tilde{w}_{ij})]. \quad (54)$$

Minimizing this objective attracts observations that are strongly connected in the original latent space and repels observations that have weak or no connections. The resulting embedding therefore preserves neighbourhood relationships rather than the exact pairwise distances of the original space.

The scale of the neighbourhood structure is controlled by the number of neighbours considered for each observation. Small values emphasize local structure, whereas larger values allow broader relationships to influence the embedding. A minimum-distance parameter further controls how closely neighbouring observations can be placed in the low-dimensional space.

In our experiments, UMAP was applied directly to the 256-dimensional latent representations using Euclidean distance and the default minimum-distance value. The neighbourhood size was varied from 5 to 55 in increments of 5, and both two- and three-dimensional embeddings were generated.

8.4 Results

8.4.1 Agglomerative clustering

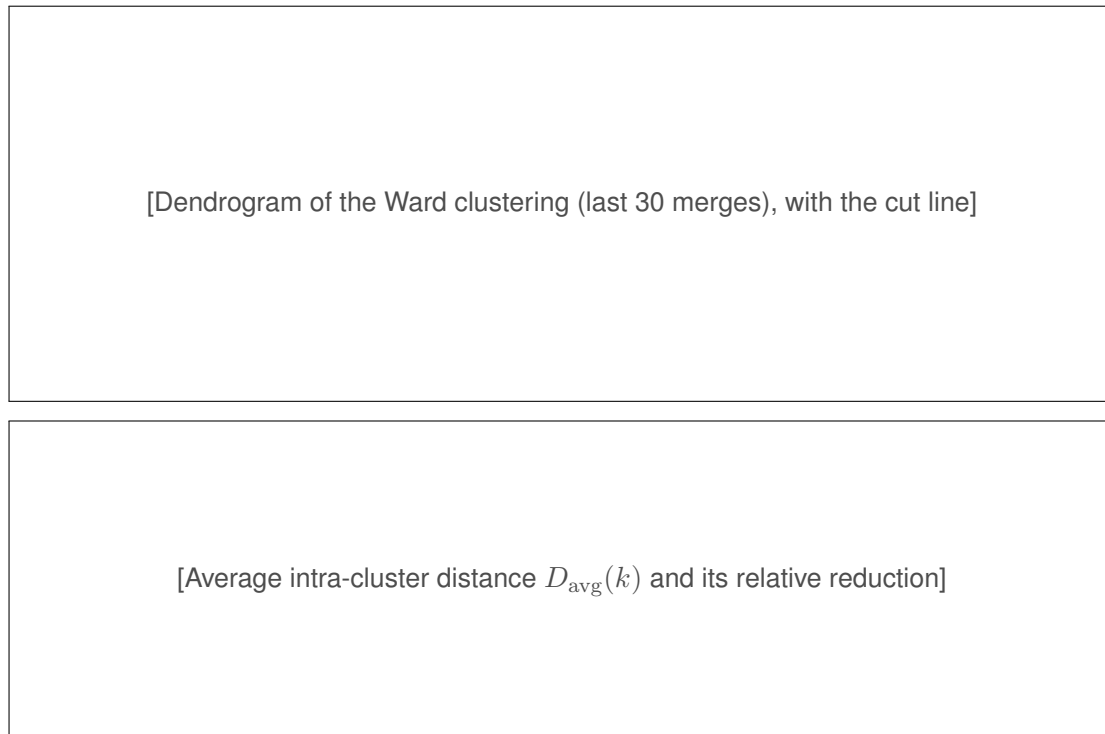


Figure 23: Agglomerative clustering of the latent codes. Top: dendrogram, cut at k^* clusters (dashed line). Bottom: average intra-cluster distance as a function of the number of clusters (Eq. (??)) and its relative reduction (Eq. (??)).

[Interpret: how many clusters are suggested, how clear is the largest drop compared with the next candidates, and what are the cluster sizes? Does the dendrogram show well-separated branches, or merges at similar heights?]

[Kernel PCA | t-SNE | UMAP, coloured by Ward cluster]

Figure 24: Latent codes projected with kernel PCA, t-SNE (perplexity **[TODO: value]**) and UMAP ($n_neighbors$ **[TODO: value]**), coloured by agglomerative cluster.

[Interpret: are the clusters separated in all three projections, only in some, or in none? Is the picture stable over the parameter sweep?]

8.4.2 K-means and kernel K-means

[K-means and kernel K-means: inertia (elbow) and mean silhouette for $K = 2, \dots, 10$]

Figure 25: Choice of the number of clusters for K-means and kernel K-means: inertia (Eq. (??)) and mean silhouette (Eq. (??)).

Table 11: Agreement between partitions, measured with the adjusted Rand index (1 = same partition, 0 = chance level).

Comparison	Latent codes	ARI
Agglomerative ($K = 4$) vs K-means ($K = 3$)	unregistered	[TODO: ?]
Agglomerative ($K = 4$) vs K-means ($K = 3$)	registered	[TODO: ?]
K-means: unregistered vs registered	–	[TODO: ?]
Agglomerative: unregistered vs registered	–	[TODO: ?]

[Interpret: which K do the elbow and the silhouette suggest, and do they agree? How much do the two methods, and the two sets of codes, agree (Table ??)? Report the robustness checks of Section “??”: seed stability (mean ARI), sensitivity to γ , and the silhouette of the real data compared with the no-cluster Gaussian baseline.]

8.4.3 Morphological characterization of the clusters

For every tree, the six descriptors of Section “??” are averaged over its branches and complemented with two whole-tree counts, the number of branches and the deepest generation. Comparing these eight values between clusters shows what distinguishes the groups, and colouring the projections by each value shows whether the latent space is organized along these anatomical properties.

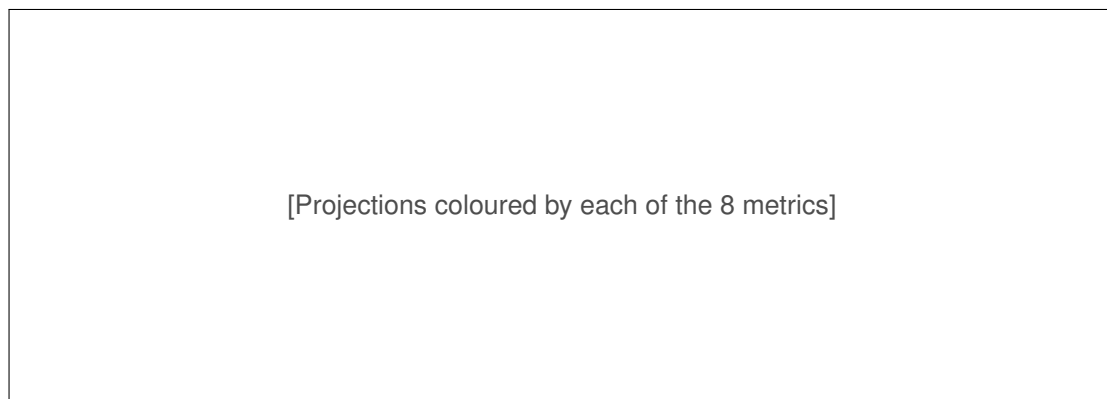


Figure 26: UMAP projection of the latent codes coloured by the mean value of each morphological descriptor and by the two tree counts.

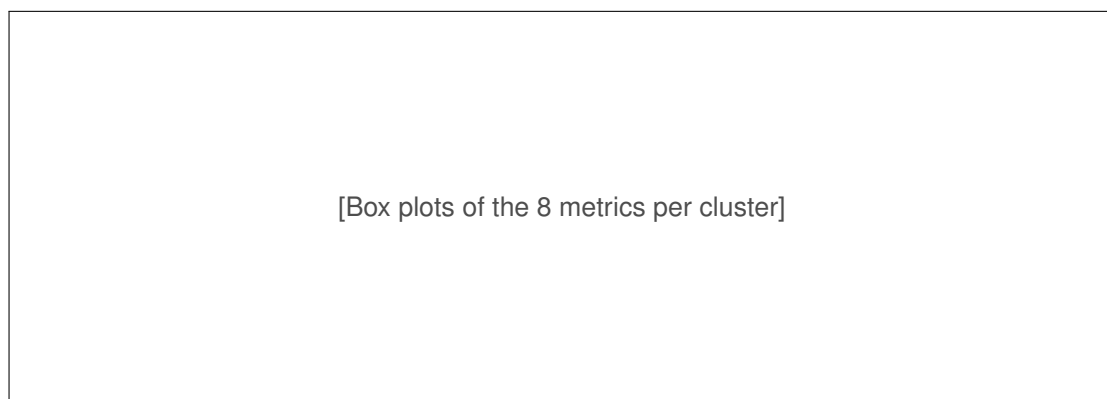


Figure 27: Distribution of the eight metrics in each cluster.

Tables ?? to ?? give, for each method and each set of latent codes, the distribution of the eight metrics within every cluster: mean, standard deviation and variance, median and interquartile range. The shading of each cell shows the standardized difference between the cluster mean and the mean over all 419 trees,

$$d = \frac{\bar{x}_{\text{cluster}} - \bar{x}_{\text{all}}}{s_{\text{all}}}, \quad (55)$$

where s_{all} is the standard deviation over all trees: red cells lie above the overall mean, blue cells below, and darker cells further from it. Expressing the difference in units of the overall spread makes metrics with very different scales comparable, from tortuosity values around 0.6 to branch counts above 200. Whether a metric differs significantly between the clusters can be tested with a Kruskal–Wallis test (?), which compares the distributions without assuming normality. **[TODO: Add the Kruskal–Wallis p -values per metric, e.g. as one sentence per table or as an extra column.]**

article [a4paper,landscape,margin=1.5cm]geometry [table]xcolor booktabs graphicx amsmath

Table 12: Cluster statistics: K-means, unregistered.

Metric	• Cluster 0 (n=140)	• Cluster 1 (n=107)	• Cluster 2 (n=172)	All
divergence	0.3799 SD 0.05221 · σ^2 0.002726	0.3645 SD 0.05863 · σ^2 0.003437	0.3955 SD 0.0451 · σ^2 0.002034	0.3824 SD 0.05258 · σ^2 0.002764
length	52.02 SD 7.471 · σ^2 55.82	48.82 SD 9.631 · σ^2 92.77	55.39 SD 6.287 · σ^2 39.53	52.58 SD 8.077 · σ^2 65.23
complexity	0.7748 SD 0.08176 · σ^2 0.006685	0.7643 SD 0.1027 · σ^2 0.01055	0.811 SD 0.06113 · σ^2 0.003737	0.787 SD 0.08274 · σ^2 0.006845
tortuosity	0.6253 SD 0.01658 · σ^2 2.75×10^{-4}	0.6029 SD 0.02305 · σ^2 5.31×10^{-4}	0.6247 SD 0.01793 · σ^2 3.21×10^{-4}	0.6193 SD 0.02123 · σ^2 4.51×10^{-4}
stenosis	0.3748 SD 0.03389 · σ^2 0.001149	0.3882 SD 0.03415 · σ^2 0.001166	0.3799 SD 0.0248 · σ^2 6.15×10^{-4}	0.3803 SD 0.03091 · σ^2 9.55×10^{-4}
ectasia	0.754 SD 0.01499 · σ^2 2.25×10^{-4}	0.7433 SD 0.01407 · σ^2 1.98×10^{-4}	0.7597 SD 0.01118 · σ^2 1.25×10^{-4}	0.7536 SD 0.01478 · σ^2 2.18×10^{-4}
n_branches	217 SD 78.65 · σ^2 6186	234.6 SD 101.7 · σ^2 10330	250 SD 69.41 · σ^2 4817	235 SD 82.76 · σ^2 6850
max_generation	10.99 SD 1.476 · σ^2 2.18	11.3 SD 1.829 · σ^2 3.344	11.36 SD 1.279 · σ^2 1.635	11.22 SD 1.506 · σ^2 2.269

Shading: $d = (\bar{x}_{\text{cluster}} - \bar{x}_{\text{all}}) / s_{\text{all}}$, clipped at $|d| = 1$; red = above overall mean, blue = below.

Table 13: Cluster statistics: Agglomerative clustering, unregistered.

Metric	• Cluster 0 (n=187)	• Cluster 1 (n=112)	• Cluster 2 (n=68)	• Cluster 3 (n=52)	All
divergence	0.3824 SD 0.05107 · σ^2 0.002608	0.4018 SD 0.03565 · σ^2 0.001271	0.3579 SD 0.06208 · σ^2 0.003854	0.3724 SD 0.05986 · σ^2 0.003583	0.3824 SD 0.05258 · σ^2 0.002764
length	52.82 SD 6.997 · σ^2 48.95	56.15 SD 5.685 · σ^2 32.32	48.37 SD 10.15 · σ^2 103.1	49.58 SD 9.452 · σ^2 89.34	52.58 SD 8.077 · σ^2 65.23
complexity	0.79 SD 0.07292 · σ^2 0.005317	0.8175 SD 0.05175 · σ^2 0.002678	0.7535 SD 0.1084 · σ^2 0.01175	0.754 SD 0.1037 · σ^2 0.01076	0.787 SD 0.08274 · σ^2 0.006845
tortuosity	0.6185 SD 0.02183 · σ^2 4.76×10^{-4}	0.6265 SD 0.01584 · σ^2 2.51×10^{-4}	0.607 SD 0.02483 · σ^2 6.17×10^{-4}	0.623 SD 0.01636 · σ^2 2.68×10^{-4}	0.6193 SD 0.02123 · σ^2 4.51×10^{-4}
stenosis	0.3816 SD 0.0283 · σ^2 8.01×10^{-4}	0.3796 SD 0.02333 · σ^2 5.44×10^{-4}	0.3829 SD 0.04191 · σ^2 0.001756	0.3739 SD 0.03687 · σ^2 0.001359	0.3803 SD 0.03091 · σ^2 9.55×10^{-4}
ectasia	0.7522 SD 0.01548 · σ^2 2.4×10^{-4}	0.7605 SD 0.009722 · σ^2 9.45×10^{-5}	0.7481 SD 0.01402 · σ^2 1.96×10^{-4}	0.7509 SD 0.01706 · σ^2 2.91×10^{-4}	0.7536 SD 0.01478 · σ^2 2.18×10^{-4}
n_branches	236 SD 81.53 · σ^2 6648	253.1 SD 58.7 · σ^2 3445	222.9 SD 104.7 · σ^2 10960	208.4 SD 91.41 · σ^2 8356	235 SD 82.76 · σ^2 6850
max_generation	11.29 SD 1.489 · σ^2 2.217	11.44 SD 1.129 · σ^2 1.275	11.25 SD 1.872 · σ^2 3.504	10.48 SD 1.565 · σ^2 2.451	11.22 SD 1.506 · σ^2 2.269

Shading: $d = (\bar{x}_{\text{cluster}} - \bar{x}_{\text{all}}) / s_{\text{all}}$, clipped at $|d| = 1$; red = above overall mean, blue = below.

Table 14: Cluster statistics: K-means, registered.

Metric	• Cluster 0 (n=151)	• Cluster 1 (n=200)	• Cluster 2 (n=68)	All
divergence	0.4032 SD 0.0438 · σ^2 0.001918	0.3676 SD 0.0522 · σ^2 0.002725	0.3794 SD 0.05677 · σ^2 0.003222	0.3824 SD 0.05258 · σ^2 0.002764
length	56.19 SD 6.06 · σ^2 36.73	49.78 SD 8.307 · σ^2 69	52.85 SD 8.182 · σ^2 66.94	52.58 SD 8.077 · σ^2 65.23
complexity	0.821 SD 0.06085 · σ^2 0.003703	0.7613 SD 0.08631 · σ^2 0.00745	0.7869 SD 0.08787 · σ^2 0.007722	0.787 SD 0.08274 · σ^2 0.006845
tortuosity	0.622 SD 0.01903 · σ^2 3.62×10^{-4}	0.6197 SD 0.02246 · σ^2 5.04×10^{-4}	0.6124 SD 0.02094 · σ^2 4.39×10^{-4}	0.6193 SD 0.02123 · σ^2 4.51×10^{-4}
stenosis	0.38 SD 0.02613 · σ^2 6.83×10^{-4}	0.3772 SD 0.03422 · σ^2 0.001171	0.3902 SD 0.02871 · σ^2 8.24×10^{-4}	0.3803 SD 0.03091 · σ^2 9.55×10^{-4}
ectasia	0.7589 SD 0.01328 · σ^2 1.76×10^{-4}	0.751 SD 0.01537 · σ^2 2.36×10^{-4}	0.7494 SD 0.01278 · σ^2 1.63×10^{-4}	0.7536 SD 0.01478 · σ^2 2.18×10^{-4}
n_branches	263.8 SD 74.84 · σ^2 5601	212.9 SD 77.67 · σ^2 6032	236.4 SD 94.54 · σ^2 8938	235 SD 82.76 · σ^2 6850
max_generation	11.53 SD 1.356 · σ^2 1.837	10.96 SD 1.49 · σ^2 2.219	11.31 SD 1.739 · σ^2 3.023	11.22 SD 1.506 · σ^2 2.269

Shading: $d = (\bar{x}_{\text{cluster}} - \bar{x}_{\text{all}}) / s_{\text{all}}$, clipped at $|d| = 1$; red = above overall mean, blue = below.

Table 15: Cluster statistics: Agglomerative clustering, registered.

Metric	• Cluster 0 (n=325)	• Cluster 1 (n=74)	• Cluster 2 (n=13)	• Cluster 3 (n=7)	All
divergence	0.3789 SD 0.05366 · σ^2 0.002879	0.3931 SD 0.04695 · σ^2 0.002205	0.3854 SD 0.04919 · σ^2 0.00242	0.4242 SD 0.03629 · σ^2 0.001317	0.3824 SD 0.05258 · σ^2 0.002764
length	52.27 SD 8.174 · σ^2 66.82	53.34 SD 6.939 · σ^2 48.15	52.42 SD 11.04 · σ^2 121.9	59.61 SD 6.027 · σ^2 36.33	52.58 SD 8.077 · σ^2 65.23
complexity	0.7822 SD 0.08453 · σ^2 0.007145	0.8042 SD 0.06708 · σ^2 0.004499	0.7707 SD 0.1059 · σ^2 0.01121	0.8591 SD 0.04858 · σ^2 0.00236	0.787 SD 0.08274 · σ^2 0.006845
tortuosity	0.6195 SD 0.02139 · σ^2 4.58×10^{-4}	0.6177 SD 0.02139 · σ^2 4.57×10^{-4}	0.6235 SD 0.0197 · σ^2 3.88×10^{-4}	0.621 SD 0.01674 · σ^2 2.8×10^{-4}	0.6193 SD 0.02123 · σ^2 4.51×10^{-4}
stenosis	0.3802 SD 0.03151 · σ^2 9.93×10^{-4}	0.3797 SD 0.02922 · σ^2 8.54×10^{-4}	0.3789 SD 0.03077 · σ^2 9.47×10^{-4}	0.3937 SD 0.02064 · σ^2 4.26×10^{-4}	0.3803 SD 0.03091 · σ^2 9.55×10^{-4}
ectasia	0.7532 SD 0.01518 · σ^2 2.3×10^{-4}	0.7541 SD 0.01261 · σ^2 1.59×10^{-4}	0.755 SD 0.0177 · σ^2 3.13×10^{-4}	0.765 SD 0.006023 · σ^2 3.63×10^{-5}	0.7536 SD 0.01478 · σ^2 2.18×10^{-4}
n_branches	230.4 SD 84.18 · σ^2 7087	252 SD 71.89 · σ^2 5167	220.5 SD 83.75 · σ^2 7013	299.3 SD 87.31 · σ^2 7624	235 SD 82.76 · σ^2 6850
max_generation	11.19 SD 1.541 · σ^2 2.375	11.38 SD 1.372 · σ^2 1.882	11 SD 1.528 · σ^2 2.333	11.57 SD 1.272 · σ^2 1.619	11.22 SD 1.506 · σ^2 2.269

Shading: $d = (\bar{x}_{\text{cluster}} - \bar{x}_{\text{all}}) / s_{\text{all}}$, clipped at $|d| = 1$; red = above overall mean, blue = below.